C++ COMPLETE REFERENCE LIBRARY • MODULE 06

06 POINTERS AND REFERENCES — Complete Theory & Study Book

Comprehensive Offline Theory, Memory Diagrams, Classroom C++ Programs, Practice Questions & Solutions

Module Table of Contents

1. **Module Overview & Study Guide** (README)
2. **Theory Chapter:** 01 pointers fundamentals and memory
3. **Theory Chapter:** 02 pointer arithmetic and 13 expressions
4. **Theory Chapter:** 03 const pointers arrays functions objects
5. **Theory Chapter:** 04 memory hazards and smart pointers
6. **Theory Chapter:** 05 references and pointer comparison
7. **Visual Memory & Concept Diagrams** (4 Vector Diagrams)
8. **Annotated C++ Source Programs** (8 Programs)
9. **Practice Question Bank:** conceptual questions
10. **Practice Question Bank:** debugging questions
11. **Practice Question Bank:** predict the output
12. **Practice Question Bank:** programming questions
13. **Step-by-Step Solutions:** conceptual solutions
14. **Step-by-Step Solutions:** output solutions
15. **Step-by-Step Solutions:** programming solutions

06 — Pointers and References (Priority Deep-Dive Module)

Pointers and references form the backbone of memory manipulation, indirect access, dynamic data structures, and runtime polymorphism in C++. This module provides an exhaustive, exam-ready and standard-accurate treatment of pointers, references, and the 13 classic pointer arithmetic/dereference expressions.

Directory Contents

1. Theory (`01_THEORY/`)

- `01_pointers_fundamentals_and_memory.md` (./01_THEORY/01_pointers_fundamentals_and_memory.md) — Definition, intuition, address-of (`&`), dereference (`*`), `nullptr` , `void*` , and pointer-to-pointer (`int**`).

- `02_pointer_arithmetic_and_13_expressions.md` (./01_THEORY/02_pointer_arithmetic_and_13_expressions.md) — **Dedicated 13-Expression Analysis** (`*p++` , `(*p)++` , `++*p` , `*++p` , `++(*p)` , `*(++p)` , `*p--` , `(*p)--` , `--*p` , `*--p` , `*(p + 1)` , `*(p - 1)` , `*p + 1`), precedence vs. sequencing rules, and why `*p++ + *p++` has Undefined Behavior.
- `03_const_pointers_arrays_functions_objects.md` (./01_THEORY/01_pointers_fundamentals_and_memory.md) — `const` pointers vs. pointers to `const` , array decay, function parameters, C-strings, and pointers to classroom objects (→).
- `04_memory_hazards_and_smart_pointers.md` (./01_THEORY/04_memory_hazards_and_smart_pointers.md) — Wild pointers, dangling pointers, memory leaks, double deletion, and modern C++ smart pointers (`std::unique_ptr` , `std::shared_ptr` , `std::weak_ptr`).
- `05_references_and_pointer_comparison.md` (./01_THEORY/05_references_and_pointer_comparison.md) — References (`int&`), aliasing rules, and a side-by-side comparison of pointers vs. references.

2. Programs (`02_PROGRAMS/`)

- `01_pointer_basics.cpp` (./02_PROGRAMS/01_pointer_basics.cpp) — Category A: Address-of, dereference, and indirect modification.
- `02_thirteen_pointer_expressions.cpp` (./02_PROGRAMS/02_thirteen_pointer_expressions.cpp) — Category A: Independent verification of all 13 pointer expressions on a known array `{10, 20, 30}` .
- `03_unsequenced_pointer_ub.cpp` (./02_PROGRAMS/03_unsequenced_pointer_ub.cpp) — Category D: Demonstration and explanation of Undefined Behavior in `*p++ + *p++` .
- `04_const_pointer_variations.cpp` (./02_PROGRAMS/04_const_pointer_variations.cpp) — Category A: Pointer to `const` , `const` pointer, and `const` pointer to `const` .
- `05_const_pointer_compile_error.cpp` (./02_PROGRAMS/05_const_pointer_compile_error.cpp) — Category B: Intentional compile-time error when mutating through `const int*` or reassigning `int* const` .
- `06_pointer_to_pointer_and_objects.cpp` (./02_PROGRAMS/06_pointer_to_pointer_and_objects.cpp) — Category A: Double pointers (`int**`) and pointers to a classroom `person` object.
- `07_references_vs_pointers.cpp` (./02_PROGRAMS/07_references_vs_pointers.cpp) — Category A: Swapping values using references vs. pointers.
- `08_smart_pointers_modern_cpp.cpp` (./02_PROGRAMS/08_smart_pointers_modern_cpp.cpp) — Category A: Classroom `person` managed safely with `std::unique_ptr` and `std::shared_ptr` .

3. Practice Questions (`03_PRACTICE_QUESTIONS/`) — 32 Graded Questions

- `conceptual_questions.md` (./03_PRACTICE_QUESTIONS/conceptual_questions.md) — Questions 1 to 10 (Levels 1–4).
- `predict_the_output.md` (./03_PRACTICE_QUESTIONS/predict_the_output.md) — Questions 11 to 20 (Levels 1–4).
- `debugging_questions.md` (./03_PRACTICE_QUESTIONS/debugging_questions.md) — Questions 21 to 26 (Levels 2–4).

- `programming_questions.md` (./03_PRACTICE_QUESTIONS/programming_questions.md) — Questions 27 to 32 (Levels 2–4).

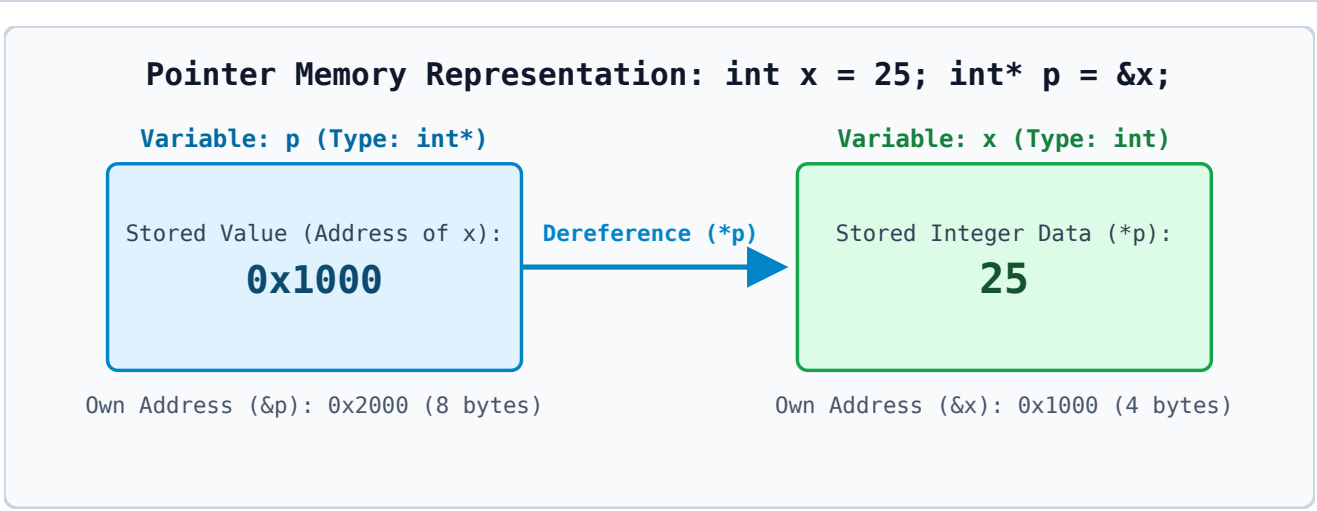
4. Detailed Solutions (`04_DETAILED_SOLUTIONS/`)

- `conceptual_solutions.md` (./04_DETAILED_SOLUTIONS/conceptual_solutions.md) — Detailed answers for Q1–Q10.
- `output_solutions.md` (./04_DETAILED_SOLUTIONS/output_solutions.md) — Step-by-step dry runs and solutions for Q11–Q26.
- `programming_solutions.md` (./04_DETAILED_SOLUTIONS/programming_solutions.md) — Complete classroom-style C++23 code and dry runs for Q27–Q32.

5. Technical Diagrams (`05_DIAGRAMS/`)

- `diagram_01_pointer_memory_model.svg` (./05_DIAGRAMS/diagram_01_pointer_memory_model.svg)
- `diagram_02_thirteen_pointer_expressions.svg` (./05_DIAGRAMS/diagram_02_thirteen_pointer_expressions.svg)
- `diagram_03_pointer_to_pointer.svg` (./05_DIAGRAMS/diagram_03_pointer_to_pointer.svg)
- `diagram_04_const_pointer_matrix.svg` (./05_DIAGRAMS/diagram_04_const_pointer_matrix.svg)

01 — Pointers: Fundamentals, Memory Model, and Indirection



Pointer Memory Model

1. Definition

A **pointer** is a strongly-typed variable that stores the **memory address** of another object or function rather than storing a direct data value.

2. Simple Explanation & Real-World Analogy

When you declare a normal integer variable:

```cpp
int x = 25;
```

The compiler reserves a block of memory (typically 4 bytes on modern 64-bit architectures), associates the name `x` with that block's starting byte address (for example, `0x1000`), and writes the binary representation of `25` inside it.

A **pointer** `p` is like an index card in a library catalog:

- The book (`x`) sits on a specific shelf (`0x1000`) and contains the actual text (`25`).
- The catalog card (`p`) sits in a catalog drawer (`0x2000`) and has `0x1000` written on it.
- **Reading** `p` tells you *where* the book is (`0x1000`).
- **Dereferencing** `*p` means walking to shelf `0x1000` and reading or changing the book itself (`25`).

3. Why Pointers Are Needed

1. **Indirect Modification Across Functions**: Passing the address of a variable allows a function to modify the caller's variable without copying large objects.

2. **Dynamic Memory Allocation**: Objects allocated on the free store (heap) via `new` have no variable name; they can only be accessed via their memory address stored in a pointer.
3. **Arrays and Strings**: Array indexing `arr[i]` is defined by the C++ standard as `*(arr + i)`.
4. **Data Structures**: Linked lists, trees, and graphs connect nodes in memory using pointers.
5. **Runtime Polymorphism**: Base-class pointers (`base*`) can point to derived-class objects (`derived`) to invoke `virtual` functions dynamically.

---

4. Core Operators: Address-of (`&`) and Dereference (`*`)

A. The Address-of Operator (`&`)

- **Syntax**: `&variable_name`
- **Operand requirement**: Must be an **lvalue** (an object with an identifiable memory location). You cannot take the address of a literal (`&42` is a compilation error) or a temporary expression (`&(a + b)` is a compilation error).
- **Result**: Returns a pointer of type `T*` (if the operand has type `T`) containing the starting byte address of the operand.

B. The Indirection / Dereference Operator (`*`)

- **Syntax**: `*pointer_name`
- **Operand requirement**: Must be a valid pointer of type `T*` (cannot be `void*`, cannot be `nullptr`, and must point to a live object).
- **Result**: Produces an **lvalue** referring to the actual object of type `T` located at the stored address.
  - On the **right side** of `=`: `int y = *p;` reads the value stored at that address.
  - On the **left side** of `=`: `*p = 99;` overwrites the value stored at that address.

---

5. Pointer Declaration and Initialization Syntax

```cpp
int x = 10;
int* p = &x;   // p is a pointer to int, initialized with the address of x
```

Understanding Every Part of the Syntax

- `int*`: The type of `p` ("pointer to `int`"). Note that in `int *p, q;`, the `*` binds to the declarator `p`, so `p` is an `int*` while `q` is a normal `int`! Always declare one pointer per line to prevent confusion.
- `p`: The identifier of the pointer variable itself.
- `= &x`: Initializes `p` so that it holds the memory address of `x`.

Why Pointers Have Types (`int*` vs `double*`)

Even though every data pointer on a 64-bit machine occupies **8 bytes** (because all 64-bit memory addresses are 8 bytes wide), the compiler must know the pointed-to type `T` for two critical reasons:

1. **Dereferencing (`*p`)**: How many bytes to read from memory (`1` byte for `char` , `4` bytes for `int` , `8` bytes for `double`) and how to interpret the bits (two's complement integer vs. IEEE-754 floating-point).
2. **Pointer Arithmetic (`p + 1`)**: How many bytes to advance the address (`sizeof(T)` bytes).

---

6. Null Pointers (`nullptr` in C++11 to C++23)

A **null pointer** is a reserved pointer state that guarantees the pointer does not point to any valid object.

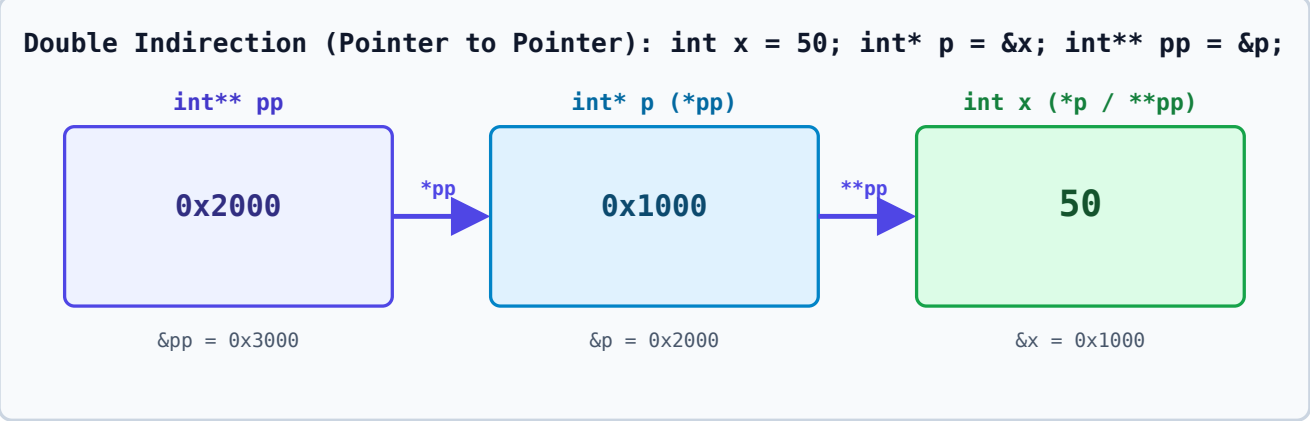
```cpp
int* p = nullptr; // Modern C++ (C++11 / C++17 / C++20 / C++23)
```

Why `nullptr` Replaced `NULL` and `0`

- In legacy C/C++, `NULL` was a preprocessor macro typically defined as `0` or `0L` (an integer literal).
- When a function was overloaded as `void func(int)` and `void func(int*)` , calling `func(NULL)` ambiguously matched or called `func(int)` instead of the pointer overload!
- `nullptr` is a keyword of type `std::nullptr_t` that converts implicitly to any pointer type (`T*`), but **never** converts to an integer type (`int`).
- **Rule**: Dereferencing a null pointer (`*p` when `p == nullptr`) is **Undefined Behavior** and typically crashes the program with a Segmentation Fault (`SIGSEGV`). Always check `if (p != nullptr)` before dereferencing a nullable pointer.

---

7. Pointer to Pointer (`int**`) and `void*`



Pointer to Pointer

A. Pointer to Pointer (`T**`)

Since a pointer `p` is itself a variable occupying memory on the stack, `p` has its own memory address (`&p`). To store the address of `p` (which has type `int*`), we need a **pointer to a pointer to** `int` (`int**`):

```cpp
int x = 50;      // Suppose &x  == 0x1000
int* p = &x;     // Suppose &p  == 0x2000, value of p  == 0x1000
int** pp = &p;   // Suppose &pp == 0x3000, value of pp == 0x2000
```

- `pp` evaluates to `0x2000` (address of `p`).
- `*pp` evaluates to `0x1000` (value of `p`, which is the address of `x`).
- `**pp` evaluates to `50` (value of `x`).

B. Generic / Void Pointer (`void*`)

A `void*` can hold the address of any non-`const`, non-`volatile` data type:

```cpp
int a = 10;
double b = 3.14;
void* vp = &a; // Valid: int* implicitly converts to void*
vp = &b;       // Valid: double* implicitly converts to void*
```

Strict Restrictions on `void*`:

1. **Cannot be dereferenced directly**: `*vp` is a **compilation error** because the compiler does not know the size or type of the target object.
2. **No pointer arithmetic**: `vp + 1` is a **compilation error** in standard C++ because `void` is an incomplete type with no size.
3. **Must be cast back first**: Use `*static_cast<double*>(vp)` to read the `double` safely.

---

8. Short Revision Summary

Expression	Meaning	Type (given `int x = 10; int* p = &x; int** pp = &p;`)	Value (assuming `&x = 0x1000`, `&p = 0x2000`)
`x`	Direct value of `x`	`int`	`10`
`&x`	Address of `x`	`int*`	`0x1000`
`p`	Address stored inside `p`	`int*`	`0x1000`
`*p`	Object at the address stored in `p`	`int` (lvalue)	`10`
`&p`	Address of the pointer variable `p` itself	`int**`	`0x2000`
`pp`	Address stored inside `pp`	`int**`	`0x2000`
`*pp`	Pointer `p` (at address stored in `pp`)	`int*` (lvalue)	`0x1000`
`**pp`	Integer `x` (double dereference)	`int` (lvalue)	`10`

02 — Pointer Arithmetic, Sequencing, and the 13 Classic Pointer Expressions

Memory State Transitions for All 13 Pointer Expressions

Baseline Before Each Expression: int arr[3] = {10, 20, 30}; int* p = &arr[1] (0x1004);

arr[0] = 10	arr[1] = 20 [Initial *p]	arr[2] = 30
Addr: 0x1000 (p - 1)	Addr: 0x1004 (p)	Addr: 0x1008 (p + 1)

# Expression	Grouping	Result	Final p Target	Final Array State
1. *p++	*(p++)	**20**	&arr[2] (0x1008)	{10, 20, 30} (unchanged)
2. (*p)++	(*p)++	**20**	&arr[1] (0x1004)	{10, 21, 30} (arr[1]++)
3. ++*p	++(*p)	**21**	&arr[1] (0x1004)	{10, 21, 30} (++arr[1])
4. *++p	*(++p)	**30**	&arr[2] (0x1008)	{10, 20, 30} (unchanged)
5. ++(*p)	++(*p)	**21**	&arr[1] (0x1004)	{10, 21, 30} (++arr[1])
6. *(++p)	*(++p)	**30**	&arr[2] (0x1008)	{10, 20, 30} (unchanged)
7. *p--	*(p--)	**20**	&arr[0] (0x1000)	{10, 20, 30} (unchanged)
8. (*p)--	(*p)--	**20**	&arr[1] (0x1004)	{10, 19, 30} (arr[1]--)
9. --*p	--(*p)	**19**	&arr[1] (0x1004)	{10, 19, 30} (--arr[1])
10. *--p	*(--p)	**10**	&arr[0] (0x1000)	{10, 20, 30} (unchanged)
11. *(p + 1)	*(p + 1)	**30**	&arr[1] (0x1004)	{10, 20, 30} (unchanged)
12. *(p - 1)	*(p - 1)	**10**	&arr[1] (0x1004)	{10, 20, 30} (unchanged)
13. *p + 1	(*p) + 1	**21**	&arr[1] (0x1004)	{10, 20, 30} (unchanged)

Thirteen Pointer Expressions Diagram

1. How Pointer Arithmetic Works Internally

When a pointer `T* p` points to an element of an array `T arr[N]`, adding or subtracting an integer `k` scales the byte offset automatically by `sizeof(T)`:

$$\text{New Address}(p + k) = \text{Address}(p) + k \times \text{sizeof}(T)$$

For example, assuming `sizeof(int) == 4` and an array `int arr[3] = {10, 20, 30};` starting at base address `0x1000`:

- `&arr[0]` (`p - 1` if `p == &arr[1]`) is `0x1000` (holds `10`).
- `&arr[1]` (`p`) is `0x1004` (holds `20`).
- `&arr[2]` (`p + 1` if `p == &arr[1]`) is `0x1008` (holds `30`).
- `&arr[3]` (one-past-the-end pointer) is `0x100C` (valid to form and compare, **Undefined Behavior** to dereference).

2. Precedence vs. Associativity vs. Evaluation Order (Crucial Distinction!)

Before analyzing the 13 expressions, you must never confuse three distinct C++ rules:

1. **Operator Precedence**: Determines which operator binds more tightly to an operand when parentheses are absent:
   - **Level 2 (Higher)**: Postfix increment `expr++` and postfix decrement `expr--` (Associativity: **Left-to-Right**).
   - **Level 3 (Lower than Level 2)**: Prefix increment `++expr`, prefix decrement `--expr`, and unary dereference `*expr` (Associativity: **Right-to-Left**).
   - **Level 6**: Binary addition `+` and subtraction `-` (Associativity: **Left-to-Right**).
2. **Operator Associativity**: Determines how operators of the **same** precedence level group when adjacent. Because unary `*` and prefix `++` both sit at Level 3 with **Right-to-Left** associativity:
   - `++*p` groups from right to left as `++(*p)`.
   - `*++p` groups from right to left as `*(++p)`.
   - By contrast, in `*p++`, postfix `++` is at Level 2 (higher than unary `*` at Level 3), so `*p++` **always** groups as `*(p++)`, **never** as `(*p)++`!
3. **Evaluation Order & Sequencing (`[intro.execution]`)**:
   - Precedence and associativity only define the **parse tree** (how subexpressions are grouped); they do **NOT** by themselves define the runtime order in which separate subexpressions are evaluated!
   - Why does `*p++` dereference the *old* pointer value even though `p++` binds tighter? Because the semantics of postfix `p++` are: **yield a temporary copy of the original pointer value `p` as the result of the subexpression `p++`**, while scheduling the side effect (`p = p + 1`). The unary `*` operator then dereferences that yielded *original* address!

---

3. Baseline Setup for All 13 Independent Expressions

To compare all 13 expressions fairly and without ambiguity, assume that **before each expression runs**, we reset an array and pointer to the following exact state:

CPP

```cpp
int arr[3] = {10, 20, 30};
int* p = &arr[1]; // p starts at index 1 (address 0x1004, pointing to 20)
int result = <EXPRESSION>;
```

- Initial array: `arr[0] = 10` (`0x1000`), `arr[1] = 20` (`0x1004`), `arr[2] = 30` (`0x1008`).
- Initial pointer: `p = &arr[1]` (`0x1004`, so `*p` is initially `20`).

---

4. Independent Analysis of All 13 Pointer Expressions

Expression 1: `*p++`

1. **Operator Precedence**: Postfix `++` (Level 2) has higher precedence than unary `*` (Level 3).

2. **Associativity**: Postfix `++` is Left-to-Right; unary `*` is Right-to-Left. Grouping is `*(p++)`.
3. **Exact Meaning**: Advance pointer `p` to the next array element, but dereference the **old** address that `p` held before incrementing.
4. **Does the pointer change?**: **YES** (`p` moves from `&arr[1]` to `&arr[2]`).
5. **Does the pointed-to value change?**: **NO** (all array elements remain `{10, 20, 30}`).
6. **Step-by-Step Execution**:
   - Step 1: `p++` evaluates to the old address `&arr[1]` (`0x1004`) and increments `p` to `&arr[2]` (`0x1008`).
   - Step 2: `*` dereferences the yielded old address `0x1004`, fetching `arr[1]` (`20`).
   - Step 3: `result` receives `20`.
7. **Dry-Run Table**:

Step	Action	`p` (Address)	`arr[0]`	`arr[1]`	`arr[2]`	`result`
Initial	`p = &arr[1]`	`0x1004` (`&arr[1]`)	`10`	`20`	`30`	—
After `result = *p++`	Dereference old `p`, increment `p`	`0x1008` (`&arr[2]`)	`10`	`20`	`30`	`20`

8. **Final Pointer Position**: `&arr[2]` (`0x1008`, pointing to `30`).
9. **Final Array Values**: `{10, 20, 30}`.

Expression 2: `(*p)++`

1. **Operator Precedence**: Parentheses `()` (Level 1) override default precedence, forcing `*p` to be evaluated first, followed by postfix `++` (Level 2) on the resulting lvalue `arr[1]`.
2. **Associativity**: N/A (explicit parentheses).
3. **Exact Meaning**: Dereference `p` to get the integer variable `arr[1]`, yield its old value (`20`), and increment that integer variable in memory (`arr[1]` becomes `21`).
4. **Does the pointer change?**: **NO** (`p` stays at `&arr[1]`).
5. **Does the pointed-to value change?**: **YES** (`arr[1]` changes from `20` to `21`).
6. **Step-by-Step Execution**:
   - Step 1: `(*p)` dereferences `p` (`&arr[1]`), producing the lvalue `arr[1]`.
   - Step 2: Postfix `++` yields the old value of `arr[1]` (`20`) and increments `arr[1]` to `21`.
   - Step 3: `result` receives `20`.
7. **Dry-Run Table**:

Step	Action	`p` (Address)	`arr[0]`	`arr[1]`	`arr[2]`	`result`
Initial	`p = &arr[1]`	`0x1004` (`&arr[1]`)	`10`	`20`	`30`	—
After `result = (*p)++`	Post-increment `arr[1]`	`0x1004` (`&arr[1]`)	`10`	`21`	`30`	`20`

8. **Final Pointer Position**: `&arr[1]` (`0x1004`, pointing to `21`).
9. **Final Array Values**: `{10, 21, 30}`.

Expression 3: `++*p`

1. **Operator Precedence**: Prefix `++` and unary `*` share the **same** precedence (Level 3).
2. **Associativity**: Level 3 operators associate **Right-to-Left**, so `++*p` groups as `++(*p)`.
3. **Exact Meaning**: Dereference `p` to obtain the lvalue `arr[1]`, pre-increment `arr[1]` from `20` to `21`, and yield the updated lvalue (`21`).
4. **Does the pointer change?**: **NO** (`p` stays at `&arr[1]`).
5. **Does the pointed-to value change?**: **YES** (`arr[1]` changes from `20` to `21`).
6. **Step-by-Step Execution**:
   - Step 1: Inner right-hand operator `*p` accesses `arr[1]`.
   - Step 2: Outer left-hand operator `++` increments `arr[1]` in place from `20` to `21` and yields `21`.
   - Step 3: `result` receives `21`.
7. **Dry-Run Table**:

Step	Action	`p` (Address)	`arr[0]`	`arr[1]`	`arr[2]`	`result`
Initial	`p = &arr[1]`	`0x1004` (`&arr[1]`)	`10`	`20`	`30`	—
After `result = ++*p`	Pre-increment `arr[1]`	`0x1004` (`&arr[1]`)	`10`	`21`	`30`	`21`

8. **Final Pointer Position**: `&arr[1]` (`0x1004`, pointing to `21`).
9. **Final Array Values**: `{10, 21, 30}`.

Expression 4: `*++p`

1. **Operator Precedence**: Unary `*` and prefix `++` share the **same** precedence (Level 3).
2. **Associativity**: **Right-to-Left**, so `*++p` groups as `*(++p)`.
3. **Exact Meaning**: Pre-increment the pointer `p` so it points to the next element (`&arr[2]`), and then dereference the **new** pointer address (`arr[2]`).
4. **Does the pointer change?**: **YES** (`p` moves from `&arr[1]` to `&arr[2]`).
5. **Does the pointed-to value change?**: **NO** (array remains `{10, 20, 30}`).

6. **Step-by-Step Execution**:
   - Step 1: `++p` increments `p` from `&arr[1]` (`0x1004`) to `&arr[2]` (`0x1008`) and yields the updated pointer `p` .
   - Step 2: `*` dereferences `0x1008` , yielding `arr[2]` (`30`).
   - Step 3: `result` receives `30` .
7. **Dry-Run Table**:

Step	Action	`p` (Address)	`arr[0]`	`arr[1]`	`arr[2]`	`result`
Initial	`p = &arr[1]`	`0x1004` (`&arr[1]`)	`10`	`20`	`30`	—
After `result = *++p`	Pre-increment `p` , then dereference	`0x1008` (`&arr[2]`)	`10`	`20`	`30`	`30`

8. **Final Pointer Position**: `&arr[2]` (`0x1008` , pointing to `30`).
9. **Final Array Values**: `{10, 20, 30}` .

Expression 5: `++(*p)`

1. **Operator Precedence**: Parentheses `()` (Level 1) explicitly group `*p` , and prefix `++` (Level 3) applies to that dereferenced lvalue.
2. **Associativity**: Identical in grouping and behavior to `++*p` (Expression 3).
3. **Exact Meaning**: Explicitly dereference `p` first (`arr[1]`), pre-increment `arr[1]` from `20` to `21` , and yield the new value `21` .
4. **Does the pointer change?**: **NO** (`p` stays at `&arr[1]`).
5. **Does the pointed-to value change?**: **YES** (`arr[1]` becomes `21`).
6. **Step-by-Step Execution**:
   - Step 1: `(*p)` yields the lvalue `arr[1]` .
   - Step 2: `++` increments `arr[1]` to `21` and returns `21` .
   - Step 3: `result` receives `21` .
7. **Dry-Run Table**:

Step	Action	`p` (Address)	`arr[0]`	`arr[1]`	`arr[2]`	`result`
Initial	`p = &arr[1]`	`0x1004` (`&arr[1]`)	`10`	`20`	`30`	—
After `result = ++(*p)`	Pre-increment `arr[1]`	`0x1004` (`&arr[1]`)	`10`	`21`	`30`	`21`

8. **Final Pointer Position**: `&arr[1]` (`0x1004` , pointing to `21`).
9. **Final Array Values**: `{10, 21, 30}` .

Expression 6: `*(++p)`

1. **Operator Precedence**: Parentheses `()` (Level 1) explicitly group `++p`, and unary `*` (Level 3) dereferences the result.
2. **Associativity**: Identical in grouping and behavior to `*++p` (Expression 4).
3. **Exact Meaning**: Pre-increment the pointer `p` to `&arr[2]` first, then dereference the updated pointer to obtain `arr[2]` (`30`).
4. **Does the pointer change?**: **YES** (`p` moves from `&arr[1]` to `&arr[2]`).
5. **Does the pointed-to value change?**: **NO** (array remains `{10, 20, 30}`).
6. **Step-by-Step Execution**:
   - Step 1: `(++p)` advances `p` to `&arr[2]` (`0x1008`).
   - Step 2: `*` dereferences `0x1008`, yielding `30`.
   - Step 3: `result` receives `30`.
7. **Dry-Run Table**:

Step	Action	`p` (Address)	`arr[0]`	`arr[1]`	`arr[2]`	`result`
Initial	`p = &arr[1]`	`0x1004` (`&arr[1]`)	`10`	`20`	`30`	—
After `result = *(++p)`	Pre-increment `p`, then dereference	`0x1008` (`&arr[2]`)	`10`	`20`	`30`	`30`

8. **Final Pointer Position**: `&arr[2]` (`0x1008`, pointing to `30`).
9. **Final Array Values**: `{10, 20, 30}`.

Expression 7: `*p--`

1. **Operator Precedence**: Postfix `--` (Level 2) has higher precedence than unary `*` (Level 3).
2. **Associativity**: Groups as `*(p--)`.
3. **Exact Meaning**: Decrement pointer `p` to the previous array element (`&arr[0]`), while dereferencing the **old** pointer address (`&arr[1]`).
4. **Does the pointer change?**: **YES** (`p` moves backward from `&arr[1]` to `&arr[0]`).
5. **Does the pointed-to value change?**: **NO** (array remains `{10, 20, 30}`).
6. **Step-by-Step Execution**:
   - Step 1: `p--` yields the old address `&arr[1]` (`0x1004`) and decrements `p` to `&arr[0]` (`0x1000`).
   - Step 2: `*` dereferences the yielded old address `0x1004`, reading `arr[1]` (`20`).
   - Step 3: `result` receives `20`.
7. **Dry-Run Table**:

Step	Action	`p` (Address)	`arr[0]`	`arr[1]`	`arr[2]`	`result`
Initial	`p = &arr[1]`	`0x1004` (`&arr[1]`)	`10`	`20`	`30`	—
After `result = *p--`	Dereference old `p`, decrement `p`	`0x1000` (`&arr[0]`)	`10`	`20`	`30`	`20`

8. **Final Pointer Position**: `&arr[0]` (`0x1000`, pointing to `10`).
9. **Final Array Values**: `{10, 20, 30}`.

Expression 8: `(*p)--`

1. **Operator Precedence**: Parentheses `()` (Level 1) force `*p` first; postfix `--` (Level 2) applies to the dereferenced lvalue `arr[1]`.
2. **Associativity**: N/A (explicit parentheses).
3. **Exact Meaning**: Dereference `p` to get `arr[1]`, yield its old value (`20`), and decrement `arr[1]` in place to `19`.
4. **Does the pointer change?**: **NO** (`p` stays at `&arr[1]`).
5. **Does the pointed-to value change?**: **YES** (`arr[1]` changes from `20` to `19`).
6. **Step-by-Step Execution**:
   - Step 1: `(*p)` accesses lvalue `arr[1]`.
   - Step 2: Postfix `--` yields `20` and decrements `arr[1]` to `19`.
   - Step 3: `result` receives `20`.
7. **Dry-Run Table**:

Step	Action	`p` (Address)	`arr[0]`	`arr[1]`	`arr[2]`	`result`
Initial	`p = &arr[1]`	`0x1004` (`&arr[1]`)	`10`	`20`	`30`	—
After `result = (*p)--`	Post-decrement `arr[1]`	`0x1004` (`&arr[1]`)	`10`	`19`	`30`	`20`

8. **Final Pointer Position**: `&arr[1]` (`0x1004`, pointing to `19`).
9. **Final Array Values**: `{10, 19, 30}`.

Expression 9: `--*p`

1. **Operator Precedence**: Prefix `--` and unary `*` share Level 3.
2. **Associativity**: **Right-to-Left**, so `--*p` groups as `--(*p)`.
3. **Exact Meaning**: Dereference `p` to get lvalue `arr[1]`, pre-decrement `arr[1]` from `20` to `19`, and yield `19`.
4. **Does the pointer change?**: **NO** (`p` stays at `&arr[1]`).
5. **Does the pointed-to value change?**: **YES** (`arr[1]` changes from `20` to `19`).

6. **Step-by-Step Execution**:
   - Step 1: `*p` accesses lvalue `arr[1]`.
   - Step 2: Prefix `--` decrements `arr[1]` to `19` and yields `19`.
   - Step 3: `result` receives `19`.
7. **Dry-Run Table**:

Step	Action	`p` (Address)	`arr[0]`	`arr[1]`	`arr[2]`	`result`
Initial	`p = &arr[1]`	`0x1004` (`&arr[1]`)	`10`	`20`	`30`	—
After `result = --*p`	Pre-decrement `arr[1]`	`0x1004` (`&arr[1]`)	`10`	`19`	`30`	`19`

8. **Final Pointer Position**: `&arr[1]` (`0x1004`, pointing to `19`).
9. **Final Array Values**: `{10, 19, 30}`.

Expression 10: `*--p`

1. **Operator Precedence**: Unary `*` and prefix `--` share Level 3.
2. **Associativity**: **Right-to-Left**, so `*--p` groups as `*(--p)`.
3. **Exact Meaning**: Pre-decrement pointer `p` from `&arr[1]` to `&arr[0]`, then dereference the new address `&arr[0]` to yield `10`.
4. **Does the pointer change?**: **YES** (`p` moves from `&arr[1]` to `&arr[0]`).
5. **Does the pointed-to value change?**: **NO** (array remains `{10, 20, 30}`).
6. **Step-by-Step Execution**:
   - Step 1: `--p` decrements `p` to `&arr[0]` (`0x1000`) and yields `p`.
   - Step 2: `*` dereferences `0x1000`, yielding `arr[0]` (`10`).
   - Step 3: `result` receives `10`.
7. **Dry-Run Table**:

Step	Action	`p` (Address)	`arr[0]`	`arr[1]`	`arr[2]`	`result`
Initial	`p = &arr[1]`	`0x1004` (`&arr[1]`)	`10`	`20`	`30`	—
After `result = *--p`	Pre-decrement `p`, then dereference	`0x1000` (`&arr[0]`)	`10`	`20`	`30`	`10`

8. **Final Pointer Position**: `&arr[0]` (`0x1000`, pointing to `10`).
9. **Final Array Values**: `{10, 20, 30}`.

Expression 11: `*(p + 1)`

1. **Operator Precedence**: Parentheses `()` (Level 1) evaluate binary addition `p + 1` (Level 6) before unary `*` (Level 3).
2. **Associativity**: N/A (parenthesized).
3. **Exact Meaning**: Compute a temporary address one element after `p` (`&arr[2]`) and dereference that temporary address **without modifying** `p`. Equivalent to `p[1]`.
4. **Does the pointer change?**: **NO** (`p` stays at `&arr[1]` because `+` does not assign back to `p`).
5. **Does the pointed-to value change?**: **NO** (array remains `{10, 20, 30}`).
6. **Step-by-Step Execution**:
   - Step 1: `p + 1` computes temporary pointer `&arr[1] + 1 == &arr[2]` (`0x1008`). `p` itself remains `0x1004`.
   - Step 2: `*` dereferences `0x1008`, yielding `arr[2]` (`30`).
   - Step 3: `result` receives `30`.
7. **Dry-Run Table**:

Step	Action	`p` (Address)	`arr[0]`	`arr[1]`	`arr[2]`	`result`
Initial	`p = &arr[1]`	`0x1004` (`&arr[1]`)	`10`	`20`	`30`	—
After `result = *(p + 1)`	Read element at `p + 1`	`0x1004` (`&arr[1]`)	`10`	`20`	`30`	`30`

8. **Final Pointer Position**: `&arr[1]` (`0x1004`, pointing to `20`).
9. **Final Array Values**: `{10, 20, 30}`.

Expression 12: `*(p - 1)`

1. **Operator Precedence**: Parentheses `()` (Level 1) evaluate binary subtraction `p - 1` (Level 6) before unary `*` (Level 3).
2. **Associativity**: N/A (parenthesized).
3. **Exact Meaning**: Compute a temporary address one element before `p` (`&arr[0]`) and dereference that temporary address **without modifying** `p`. Equivalent to `p[-1]`.
4. **Does the pointer change?**: **NO** (`p` stays at `&arr[1]`).
5. **Does the pointed-to value change?**: **NO** (array remains `{10, 20, 30}`).
6. **Step-by-Step Execution**:
   - Step 1: `p - 1` computes temporary pointer `&arr[1] - 1 == &arr[0]` (`0x1000`). `p` remains `0x1004`.
   - Step 2: `*` dereferences `0x1000`, yielding `arr[0]` (`10`).
   - Step 3: `result` receives `10`.
7. **Dry-Run Table**:

Step	Action	`p` (Address)	`arr[0]`	`arr[1]`	`arr[2]`	`result`
Initial	`p = &arr[1]`	`0x1004` (`&arr[1]`)	`10`	`20`	`30`	—
After `result = *(p - 1)`	Read element at `p - 1`	`0x1004` (`&arr[1]`)	`10`	`20`	`30`	`10`

8. **Final Pointer Position**: `&arr[1]` (`0x1004`, pointing to `20`).
9. **Final Array Values**: `{10, 20, 30}`.

Expression 13: `*p + 1`

1. **Operator Precedence**: Unary `*` (Level 3) has higher precedence than binary `+` (Level 6).
2. **Associativity**: Groups as `(*p) + 1`.
3. **Exact Meaning**: Dereference `p` to read the integer value `arr[1]` (`20`), and add `1` to that integer value (`20 + 1 == 21`) **without modifying `p` or `arr[1]`**.
4. **Does the pointer change?**: **NO** (`p` stays at `&arr[1]`).
5. **Does the pointed-to value change?**: **NO** (`arr[1]` stays `20` because `+ 1` produces a temporary prvalue; it does not assign back to `*p`).
6. **Step-by-Step Execution**:
   - Step 1: `*p` reads `arr[1]` (`20`).
   - Step 2: `20 + 1` computes temporary integer `21`.
   - Step 3: `result` receives `21`.
7. **Dry-Run Table**:

Step	Action	`p` (Address)	`arr[0]`	`arr[1]`	`arr[2]`	`result`
Initial	`p = &arr[1]`	`0x1004` (`&arr[1]`)	`10`	`20`	`30`	—
After `result = *p + 1`	Read `*p` and add `1`	`0x1004` (`&arr[1]`)	`10`	`20`	`30`	`21`

8. **Final Pointer Position**: `&arr[1]` (`0x1004`, pointing to `20`).
9. **Final Array Values**: `{10, 20, 30}`.

---

5. Master Summary Table of All 13 Expressions

(Initial state before each row: `int arr[3] = {10, 20, 30}; int* p = &arr[1];`)

#	Expression	Effective Grouping	Pointer `p` Changes?	Array Value Changes?	Final `p` Target	Final `arr` State	Returned `result`
1	`*p++`	`*(p++)`	**Yes** (`&arr[2]`)	No	`arr[2]` (`30`)	`{10, 20, 30}`	**20**
2	`(*p)++`	`(*p)++`	No (`&arr[1]`)	**Yes** (`arr[1]=21`)	`arr[1]` (`21`)	`{10, 21, 30}`	**20**
3	`++*p`	`++(*p)`	No (`&arr[1]`)	**Yes** (`arr[1]=21`)	`arr[1]` (`21`)	`{10, 21, 30}`	**21**
4	`*++p`	`*(++p)`	**Yes** (`&arr[2]`)	No	`arr[2]` (`30`)	`{10, 20, 30}`	**30**
5	`++(*p)`	`++(*p)`	No (`&arr[1]`)	**Yes** (`arr[1]=21`)	`arr[1]` (`21`)	`{10, 21, 30}`	**21**
6	`*(++p)`	`*(++p)`	**Yes** (`&arr[2]`)	No	`arr[2]` (`30`)	`{10, 20, 30}`	**30**
7	`*p--`	`*(p--)`	**Yes** (`&arr[0]`)	No	`arr[0]` (`10`)	`{10, 20, 30}`	**20**
8	`(*p)--`	`(*p)--`	No (`&arr[1]`)	**Yes** (`arr[1]=19`)	`arr[1]` (`19`)	`{10, 19, 30}`	**20**
9	`--*p`	`--(*p)`	No (`&arr[1]`)	**Yes** (`arr[1]=19`)	`arr[1]` (`19`)	`{10, 19, 30}`	**19**
10	`*--p`	`*(--p)`	**Yes** (`&arr[0]`)	No	`arr[0]` (`10`)	`{10, 20, 30}`	**10**
11	`*(p + 1)`	`*(p + 1)`	No (`&arr[1]`)	No	`arr[1]` (`20`)	`{10, 20, 30}`	**30**
12	`*(p - 1)`	`*(p - 1)`	No (`&arr[1]`)	No	`arr[1]` (`20`)	`{10, 20, 30}`	**10**
13	`*p + 1`	`(*p) + 1`	No (`&arr[1]`)	No	`arr[1]` (`20`)	`{10, 20, 30}`	**21**

6. Why `*p++ + *p++` is Undefined Behavior (Sequencing Rules)

Students frequently see university trick questions like:

CPP

```cpp
int arr[3] = {10, 20, 30};
int* p = &arr[0];
int x = *p++ + *p++; // WARNING: UNDEFINED BEHAVIOR!
```

The Exact C++ Standard Rule (`[intro.execution]`)

1. The binary addition operator `+` does **not** impose a sequencing order between the evaluation of its left operand (`*p++`) and its right operand (`*p++`).
2. Both operands modify the same scalar object `p` (via the side effect of `p++`).
3. According to ISO C++ (`[intro.execution]`): *"If a side effect on a memory location is unsequenced relative to either another side effect on the same memory location or a value computation using the value of any object in the same memory location, and they are not potentially concurrent, the behavior is undefined."*
4. Even in C++17, C++20, and C++23 (which tightened evaluation order for `a << b` , `a.b` , `a[b]` , and assignment right-to-left), the operands of binary `+` , `-` , `*` , `/` remain **unsequenced** relative to each other!
5. **Exam Rule**: Never state a single "guaranteed" numerical output (`20` or `30`) as standard C++ behavior for `*p++ + *p++` . Always write: **"The expression exhibits Undefined Behavior (UB) due to unsequenced side effects on the scalar pointer `p` within the binary `+` expression."**

03 — `const` Pointers, Arrays, Functions, Strings, and Objects

The Right-to-Left Reading Rule for const and Pointers

`const int * p`	`p = &y (ALLOWED) | *p = 99 (COMPILE ERROR: data is const)`
`int * const p`	`p = &y (COMPILE ERROR) | *p = 99 (ALLOWED: data is mutable)`
`const int * const p`	`p = &y (COMPILE ERROR) | *p = 99 (COMPILE ERROR: both locked)`

Const Pointer Matrix

1. The Three Combinations of `const` and Pointers

Read pointer declarations **from right to left** (the "Clockwise / Right-to-Left Rule", splitting at the `*`):

A. Pointer to Constant Data (`const int* p` or `int const* p`)

- **Reading right-to-left**: `p` is a pointer (`*`) to an `int` that is `const` .
- **What is locked?**: The **pointed-to value** (`*p = 50;` is a **Compilation Error**).
- **What is mutable?**: The **pointer address itself** (`p = &other_var;` is **Allowed**).

B. Constant Pointer to Mutable Data (`int* const p`)

- **Reading right-to-left**: `p` is a `const` pointer (`* const`) to a mutable `int` .
- **What is locked?**: The **pointer address itself** (`p = &other_var;` is a **Compilation Error**; `p` must be initialized when declared).
- **What is mutable?**: The **pointed-to value** (`*p = 50;` is **Allowed**).

C. Constant Pointer to Constant Data (`const int* const p`)

- **Reading right-to-left**: `p` is a `const` pointer (`* const`) to an `int` that is `const` .
- **What is locked?**: **Both** the pointer address (`p = &other_var;` fails) and the pointed-to value (`*p = 50;` fails).

Declaration	Can reassign `p = &y`?	Can modify `*p = 99`?	Common Use Case
`int* p`	Yes	Yes	Mutating both target and which object is tracked
`const int* p`	**Yes**	**No**	Iterating through read-only array elements
`int* const p`	**No**	**Yes**	Fixed hardware register or permanent buffer handle
`const int* const p`	**No**	**No**	Fixed read-only lookup address

2. Pointers and Arrays

When an array is declared:

```cpp
int arr[4] = {10, 20, 30, 40};
```

In almost all expressions, the array identifier `arr` **decays** implicitly into a pointer to its first element (`&arr[0]` of type `int*`).

- `arr[i]` is defined by the C++ compiler as syntactic sugar for `*(arr + i)`.
- **Exceptions where `arr` does NOT decay to a pointer**:
  1. `sizeof(arr)`: Returns the size of the entire array (`4 * sizeof(int) = 16` bytes), whereas `sizeof(p)` returns the size of a pointer (`8` bytes on 64-bit systems).
  2. `&arr`: Returns a pointer to the entire 4-element array (`int (*)[4]`), so `&arr + 1` steps past all 4 elements (`+16` bytes)!
- **Crucial Distinction**: An array name `arr` is **not** a modifiable lvalue pointer; writing `arr++` or `arr = p` is a **compilation error**.

3. Pointers and Functions (Call by Address)

When you pass a pointer to a function, the function receives a copy of the **memory address**, allowing it to dereference that address and mutate the caller's original variable:

```cpp
void update_age(int* ptr) {
    if (ptr ≠ nullptr) {
        *ptr = 25; // Modifies caller's variable directly in memory
    }
}
```

4. Pointers and Strings

1. **C-Style String Literal Pointer (`const char*`)**:

   ```cpp
   const char* msg = "Hello";
   ```

   String literals are stored in read-only static memory (`const char[6]`). In modern C++ (since C++11 through C++23), assigning a string literal to a non-const `char*` (`char* s = "Hello";`) is **ill-formed (Compilation Error)** because mutating a string literal is Undefined Behavior.

2. **Modern `std::string`**:
   Prefer `std::string` (`#include <string>`) for safe, automatic memory management, bounds checking (`.at()`), and length tracking (`.length()`).

---

5. Pointers and Classroom Objects (`→` Member Access Operator)

Using our classroom `person` class:

```cpp
person p;
person* ptr = &p;

ptr→input(20);   // Equivalent to (*ptr).input(20);
ptr→display();   // Equivalent to (*ptr).display();
```

Why are parentheses required in `(*ptr).display()` ? Because the dot member-access operator `.` (Precedence Level 2) has **higher** precedence than unary `*` (Precedence Level 3). Writing `*ptr.display()` would be parsed as `*(ptr.display())` , which fails to compile because `ptr` is a pointer, not a class object! The arrow operator `ptr→display()` was invented specifically as clean syntax for `(*ptr).display()` .

04 — Memory Hazards (`new`/`delete` Pitfalls) and Modern Smart Pointers

1. The Four Classic Pointer & Memory Hazards

A. Wild Pointers (Uninitialized Pointers)

A pointer declared without initialization contains arbitrary garbage bits left on the stack:

```cpp
int* p;      // Wild pointer! Contains indeterminate address
*p = 10;     // UNDEFINED BEHAVIOR (dereferencing indeterminate address)
```

Prevention: Always initialize every pointer at declaration (`int* p = nullptr;` or `int* p = &x;`).

B. Dangling Pointers

A pointer that once pointed to a valid object, but the object's lifetime has ended (either because a local stack variable went out of scope, or because `delete` was called on heap memory):

```cpp
int* p = new int(42);
delete p;    // Memory is returned to free store, but p still holds old address!
// p is now a DANGLING POINTER
*p = 10;     // UNDEFINED BEHAVIOR (use-after-free)
p = nullptr; // Good practice with raw pointers: reset immediately after delete
```

Another classic dangling pointer mistake is returning the address of a local stack variable from a function (`return &local_var;`).

C. Memory Leaks

Allocating dynamic memory with `new` and losing the pointer address (by overwriting the pointer or letting it go out of scope) without calling `delete`:

```cpp
void leak_example() {
    int* p = new int(100);
    // Function returns without 'delete p;' → 4 bytes leaked permanently until process exit
}
```

D. Double Deletion

Calling `delete` twice on the same heap address corrupts the heap allocator's internal free list and results in **Undefined Behavior**:

```cpp
int* p = new int(50);
delete p;
delete p; // UNDEFINED BEHAVIOR (Double Free)
```

---

2. Classroom Approach vs. Modern C++ Smart Pointers (`<memory>`)

Step 1: Classroom Raw Pointer Approach

In introductory lab exams, dynamic object creation is taught using `new` and `delete`:

```cpp
person* p = new person;
p->input(21);
p->display();
delete p;
```

Limitation: If an exception is thrown or an early `return` happens before `delete p;`, the `person` object is leaked.

Step 2: Modern C++ Approach (`std::unique_ptr` , `std::shared_ptr` , `std::weak_ptr`)

Modern C++ (C++11/14/17/20/23) wraps raw pointers inside RAII class templates defined in `<memory>` that automatically `delete` the managed object in their destructor:

1. `std::unique_ptr<T>` **(Exclusive Ownership — C++11,** `std::make_unique` **in C++14)**:
   - Exactly **one** `unique_ptr` owns the heap object.
   - Cannot be copied (`p2 = p1` is a compilation error), only moved (`std::move(p1)`).
   - Zero size or speed overhead compared to a raw pointer.

   ```cpp
   auto p = std::make_unique<person>();
   p->input(21);
   p->display();
   // Automatically deleted when p goes out of scope!
   ```

2. `std::shared_ptr<T>` **(Shared Reference-Counted Ownership — C++11)**:
   - Multiple `shared_ptr` instances can co-own the same heap object.
   - Maintains an atomic **reference count** in a control block.
   - The object is destroyed only when the *last* `shared_ptr` owning it is destroyed or reset.
3. `std::weak_ptr<T>` **(Non-Owning Observer — C++11)**:
   - Observes an object managed by a `shared_ptr` **without** incrementing its strong reference count.
   - Used to break circular reference cycles (`A` holds `shared_ptr<B>` and `B` holds `weak_ptr<A>`).
   - Must call `.lock()` to obtain a temporary `shared_ptr` before accessing the object.

05 — References (`T&`) and Pointers vs. References Comparison

1. Definition of a Reference

A **reference** (`T&`) is an **alias** (an alternative name) for an already-existing object. Once initialized to refer to a variable, every operation performed on the reference acts directly on the original variable.

CPP

```cpp
int x = 10;
int& ref = x; // ref is an alias for x

ref = 25;     // Modifies x directly! x is now 25
cout << (&x == &ref); // Prints 1 (true) — ref has the exact same address as x
```

2. Strict Rules of References in C++

1. **Mandatory Initialization**: A reference **must** be initialized when declared. Writing `int& ref;` is a **Compilation Error**.
2. **Cannot Be Rebound**: Unlike a pointer, a reference cannot be made to refer to a different variable after initialization. If you write:

   CPP

   ```cpp
   int a = 10, b = 20;
   int& ref = a;
   ref = b; // Does NOT rebind ref to b! It assigns the value of b (20) into a!
   ```

3. **No Null References**: There is no such thing as a valid "null reference" (`int& r = *nullptr;` is Undefined Behavior).
4. **No References to References or Arrays of References**: You cannot declare `int& arr[5]` or a pointer to a reference `int&* p` because references are aliases, not distinct addressable objects in the C++ type system (`[dcl.ref]`).
5. **`const` References (`const T&`) Can Bind to Temporaries (Rvalues)**:
   - A non-const lvalue reference `int& r = 50;` fails to compile.
   - A `const` lvalue reference `const int& r = 50;` is **valid** and extends the lifetime of the temporary value `50` to match the scope of `r` .

3. Comprehensive Comparison: Pointers vs. References (Exam Table)

Feature	Pointer (`int* p`)	Reference (`int& ref`)
Core Concept	Stores the memory address of an object	Acts as an alias (another name) for an existing object
Declaration without Initialization	Allowed (`int* p;` — though creates a wild pointer)	**Compilation Error** (`int& ref;` is illegal)
Rebinding / Reassignment	Can be reassigned to point to different objects (`p = &y;`)	**Cannot** be rebound; stays bound to its initial target for life
Nullability	Can be `nullptr`	Cannot be null in a well-defined program
Dereferencing Syntax	Requires explicit `*p` or `p→member`	Automatic; used directly like a normal variable (`ref` , `ref.member`)
Address-of Operator (`&`)	`&p` gives the address of the pointer variable itself	`&ref` gives the address of the referred-to target variable
Indirection Levels	Supports multi-level pointers (`int**` , `int**x`)	Single level only (reference to reference collapses to `int&`)
Pointer Arithmetic	Supported (`p++` , `p + 1` , `p - q`)	Not applicable (`ref++` increments the target's value)
Arrays	Array of pointers is legal (`int* arr[10];`)	Array of references is **illegal** (`int& arr[10];` fails)
When to Prefer	Optional parameters (`nullptr`), dynamic arrays, rebindable links	Function parameters (`const T&` , `T&`), operator overloading (`operator+` , `operator<<`)

Visual Concept & Memory Diagrams (06 POINTERS AND REFERENCES)

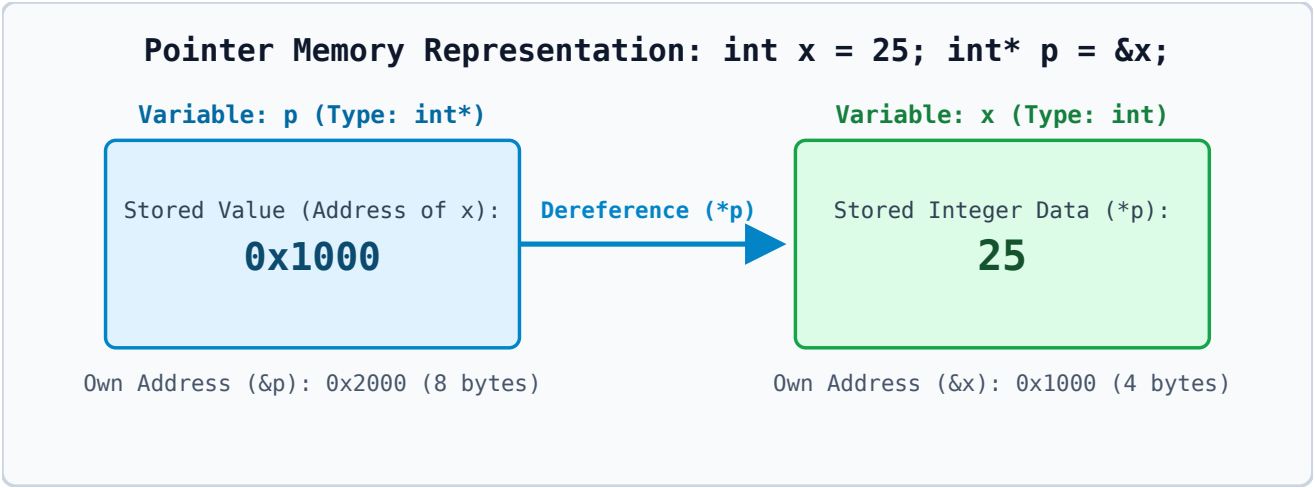


Diagram: diagram 01 pointer memory model

Memory State Transitions for All 13 Pointer Expressions

Baseline Before Each Expression: int arr[3] = {10, 20, 30}; int* p = &arr[1] (0x1004);

arr[0] = 10	arr[1] = 20 [Initial *p]	arr[2] = 30
Addr: 0x1000 (p - 1)	Addr: 0x1004 (p)	Addr: 0x1008 (p + 1)

#	Expression	Grouping	Result	Final p Target	Final Array State
1.	*p++	*(p++)	**20**	&arr[2] (0x1008)	{10, 20, 30} (unchanged)
2.	(*p)++	(*p)++	**20**	&arr[1] (0x1004)	{10, 21, 30} (arr[1]++)
3.	++*p	++(*p)	**21**	&arr[1] (0x1004)	{10, 21, 30} (++arr[1])
4.	*++p	*(++p)	**30**	&arr[2] (0x1008)	{10, 20, 30} (unchanged)
5.	++(*p)	++(*p)	**21**	&arr[1] (0x1004)	{10, 21, 30} (++arr[1])
6.	*(++p)	*(++p)	**30**	&arr[2] (0x1008)	{10, 20, 30} (unchanged)
7.	*p--	*(p--)	**20**	&arr[0] (0x1000)	{10, 20, 30} (unchanged)
8.	(*p)--	(*p)--	**20**	&arr[1] (0x1004)	{10, 19, 30} (arr[1]--)
9.	--*p	--(*p)	**19**	&arr[1] (0x1004)	{10, 19, 30} (--arr[1])
10.	*--p	*(--p)	**10**	&arr[0] (0x1000)	{10, 20, 30} (unchanged)
11.	*(p + 1)	*(p + 1)	**30**	&arr[1] (0x1004)	{10, 20, 30} (unchanged)
12.	*(p - 1)	*(p - 1)	**10**	&arr[1] (0x1004)	{10, 20, 30} (unchanged)
13.	*p + 1	(*p) + 1	**21**	&arr[1] (0x1004)	{10, 20, 30} (unchanged)

Diagram: diagram 02 thirteen pointer expressions

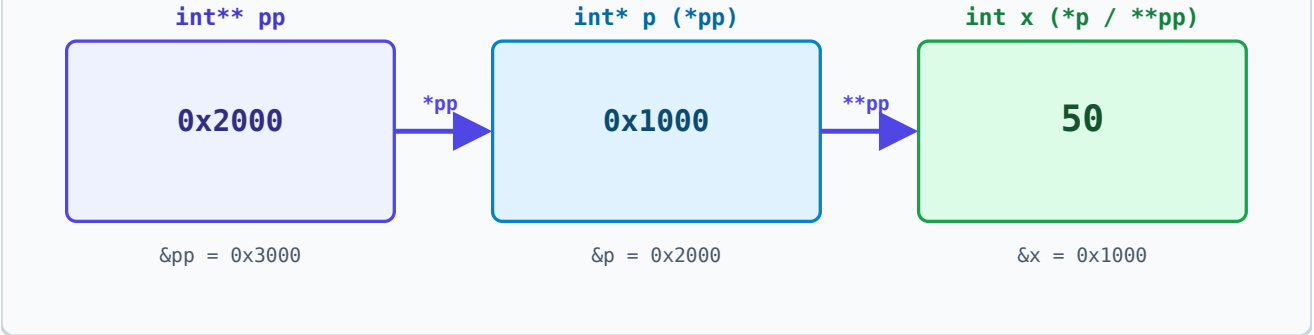


Diagram: diagram 03 pointer to pointer

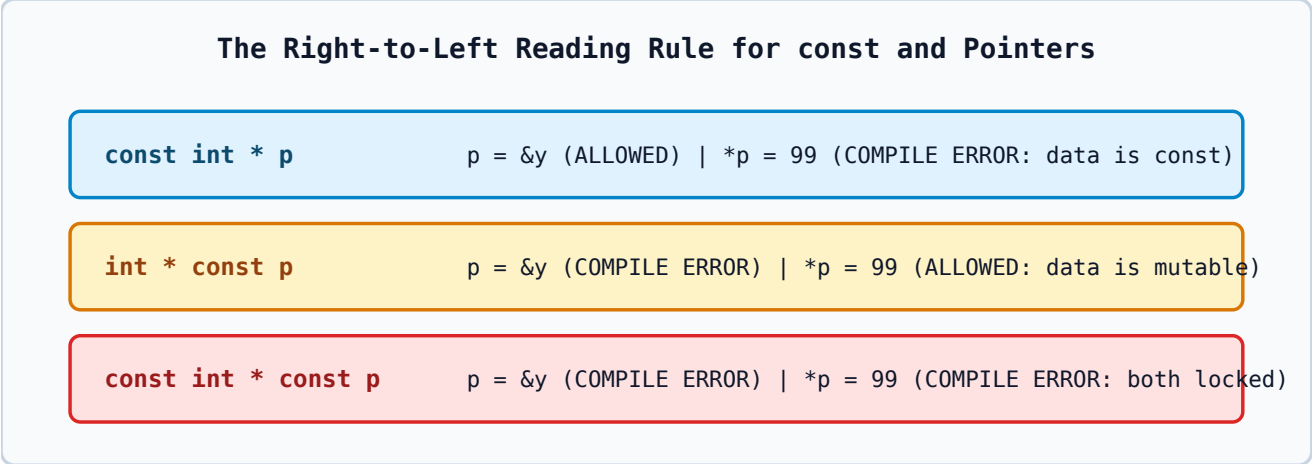


Diagram: diagram 04 const pointer matrix

Annotated C++ Programs (06 POINTERS AND REFERENCES)

Program: `01_pointer_basics.cpp`

C++ SOURCE FILE • 01_pointer_basics.cpp

```cpp
// Program Title: Basic Pointer Declaration, Address-of, and Dereferencing
// Topic: Pointers and References
// Purpose: Demonstrate basic pointer initialization (&), reading (*p), and indirect
modification (*p = value).
// Target C++ Standard: C++23 (compatible with C++17/C++20)
// Related Theory File: ../01_THEORY/01_pointers_fundamentals_and_memory.md
// Program Classification: Category A — Valid and well-defined program

#include <iostream>
using namespace std;

int main() {
    int age = 20;
    int* p = &age;

    cout << "Initial value of age: " << age << endl;
    cout << "Value accessed via *p: " << *p << endl;

    // Modify 'age' indirectly through the pointer 'p'
    *p = 25;

    cout << "Updated value of age after *p = 25: " << age << endl;
    cout << "Updated value via *p: " << *p << endl;

    return 0;
}

/*
Expected Deterministic Output:
Initial value of age: 20
Value accessed via *p: 20
Updated value of age after *p = 25: 25
Updated value via *p: 25
*/
```

Program: `02_thirteen_pointer_expressions.cpp`

C++ SOURCE FILE • 02_thirteen_pointer_expressions.cpp

```cpp
// Program Title: Independent Verification of the 13 Pointer Arithmetic & Dereference Expressions
// Topic: Pointers and References — Pointer Expression Analysis
// Purpose: Demonstrate the exact behavior of *p++, (*p)++, ++*p, *++p, ++(*p), *(++p), *p--, (*p)--, --*p, *--p, *(p+1), *(p-1), and *p+1.
// Target C++ Standard: C++23 (compatible with C++17/C++20)
// Related Theory File: ../01_THEORY/02_pointer_arithmetic_and_13_expressions.md
// Program Classification: Category A — Valid and well-defined program

#include <iostream>
using namespace std;

int main() {
    // 1. Expression: *p++
    {
        int arr[3] = {10, 20, 30};
        int* p = &arr[1];
        int res = *p++;
        cout << "1. *p++    → res=" << res << ", *p=" << *p
             << ", arr={" << arr[0] << "," << arr[1] << "," << arr[2] << "}" << endl;
    }

    // 2. Expression: (*p)++
    {
        int arr[3] = {10, 20, 30};
        int* p = &arr[1];
        int res = (*p)++;
        cout << "2. (*p)++  → res=" << res << ", *p=" << *p
             << ", arr={" << arr[0] << "," << arr[1] << "," << arr[2] << "}" << endl;
    }

    // 3. Expression: ++*p
    {
        int arr[3] = {10, 20, 30};
        int* p = &arr[1];
        int res = ++*p;
        cout << "3. ++*p    → res=" << res << ", *p=" << *p
             << ", arr={" << arr[0] << "," << arr[1] << "," << arr[2] << "}" << endl;
    }

    // 4. Expression: *++p
    {
        int arr[3] = {10, 20, 30};
        int* p = &arr[1];
        int res = *++p;
        cout << "4. *++p    → res=" << res << ", *p=" << *p
             << ", arr={" << arr[0] << "," << arr[1] << "," << arr[2] << "}" << endl;
    }

    // 5. Expression: ++(*p)
    {
        int arr[3] = {10, 20, 30};
        int* p = &arr[1];
```

```
    int res = ++(*p);
    cout << "5. ++(*p)  → res=" << res << ", *p=" << *p
         << ", arr={" << arr[0] << "," << arr[1] << "," << arr[2] << "}" << endl;
}

// 6. Expression: *(++p)
{
    int arr[3] = {10, 20, 30};
    int* p = &arr[1];
    int res = *(++p);
    cout << "6. *(++p)  → res=" << res << ", *p=" << *p
         << ", arr={" << arr[0] << "," << arr[1] << "," << arr[2] << "}" << endl;
}

// 7. Expression: *p--
{
    int arr[3] = {10, 20, 30};
    int* p = &arr[1];
    int res = *p--;
    cout << "7. *p--    → res=" << res << ", *p=" << *p
         << ", arr={" << arr[0] << "," << arr[1] << "," << arr[2] << "}" << endl;
}

// 8. Expression: (*p)--
{
    int arr[3] = {10, 20, 30};
    int* p = &arr[1];
    int res = (*p)--;
    cout << "8. (*p)--  → res=" << res << ", *p=" << *p
         << ", arr={" << arr[0] << "," << arr[1] << "," << arr[2] << "}" << endl;
}

// 9. Expression: --*p
{
    int arr[3] = {10, 20, 30};
    int* p = &arr[1];
    int res = --*p;
    cout << "9. --*p    → res=" << res << ", *p=" << *p
         << ", arr={" << arr[0] << "," << arr[1] << "," << arr[2] << "}" << endl;
}

// 10. Expression: *--p
{
    int arr[3] = {10, 20, 30};
    int* p = &arr[1];
    int res = *--p;
    cout << "10. *--p   → res=" << res << ", *p=" << *p
         << ", arr={" << arr[0] << "," << arr[1] << "," << arr[2] << "}" << endl;
}

// 11. Expression: *(p + 1)
{
    int arr[3] = {10, 20, 30};
    int* p = &arr[1];
    int res = *(p + 1);
    cout << "11. *(p+1) → res=" << res << ", *p=" << *p
         << ", arr={" << arr[0] << "," << arr[1] << "," << arr[2] << "}" << endl;
}
```

```cpp
    // 12. Expression: *(p - 1)
    {
        int arr[3] = {10, 20, 30};
        int* p = &arr[1];
        int res = *(p - 1);
        cout << "12. *(p-1) → res=" << res << ", *p=" << *p
             << ", arr={" << arr[0] << "," << arr[1] << "," << arr[2] << "}" << endl;
    }

    // 13. Expression: *p + 1
    {
        int arr[3] = {10, 20, 30};
        int* p = &arr[1];
        int res = *p + 1;
        cout << "13. *p+1   → res=" << res << ", *p=" << *p
             << ", arr={" << arr[0] << "," << arr[1] << "," << arr[2] << "}" << endl;
    }

    return 0;
}

/*
Expected Deterministic Output:
1. *p++    → res=20, *p=30, arr={10,20,30}
2. (*p)++  → res=20, *p=21, arr={10,21,30}
3. ++*p    → res=21, *p=21, arr={10,21,30}
4. *++p    → res=30, *p=30, arr={10,20,30}
5. ++(*p)  → res=21, *p=21, arr={10,21,30}
6. *(++p)  → res=30, *p=30, arr={10,20,30}
7. *p--    → res=20, *p=10, arr={10,20,30}
8. (*p)--  → res=20, *p=19, arr={10,19,30}
9. --*p    → res=19, *p=19, arr={10,19,30}
10. *--p   → res=10, *p=10, arr={10,20,30}
11. *(p+1) → res=30, *p=20, arr={10,20,30}
12. *(p-1) → res=10, *p=20, arr={10,20,30}
13. *p+1   → res=21, *p=20, arr={10,20,30}
*/
```

Program: `03_unsequenced_pointer_ub.cpp`

C++ SOURCE FILE • 03_unsequenced_pointer_ub.cpp

```cpp
// Program Title: Unsequenced Pointer Modification Undefined Behavior (*p++ + *p++)
// Topic: Pointers and References — Sequencing vs. Operator Precedence
// Purpose: Explain why *p++ + *p++ causes Undefined Behavior under ISO C++ [intro.execution]
and how to sequence it properly.
// Target C++ Standard: C++23 (compatible with C++17/C++20)
// Related Theory File: ../01_THEORY/02_pointer_arithmetic_and_13_expressions.md
// Program Classification: Category D — Undefined-behavior demonstration

#include <iostream>
using namespace std;

int main() {
    int arr[3] = {10, 20, 30};
    int* p = &arr[0];

    // UNDEFINED BEHAVIOR STATEMENT:
    // Both operands of binary '+' modify the scalar pointer 'p' via post-increment.
    // In C++ (including C++17, C++20, and C++23), the operands of '+' are unsequenced
    // relative to each other. Therefore, '*p++ + *p++' has Undefined Behavior!
    int ub_sum = *p++ + *p++;
    cout << "Result of unsequenced expression (NOT guaranteed by C++ standard): " << ub_sum
<< endl;

    // WELL-DEFINED SEQUENCED ALTERNATIVE:
    // Separate the two pointer increments across full statements (separated by ';')
    int* q = &arr[0];
    int first_val = *q++;
    int second_val = *q++;
    int defined_sum = first_val + second_val; // Guaranteed: 10 + 20 = 30
    cout << "Well-defined sequenced sum: " << defined_sum << endl;

    return 0;
}

/*
Note on Output (Category D):
No guaranteed runtime output exists for 'ub_sum' because unsequenced scalar modification
is Undefined Behavior under ISO C++ [intro.execution]. The well-defined sequenced
alternative always computes defined_sum = 30.
*/
```

Program: `04_const_pointer_variations.cpp`

C++ SOURCE FILE • 04_const_pointer_variations.cpp

```cpp
// Program Title: Const Qualifications with Pointers (Valid Operations)
// Topic: Pointers and References — const Correctness
// Purpose: Demonstrate valid operations on const int*, int* const, and const int* const.
// Target C++ Standard: C++23 (compatible with C++17/C++20)
// Related Theory File: ../01_THEORY/03_const_pointers_arrays_functions_objects.md
// Program Classification: Category A — Valid and well-defined program

#include <iostream>
using namespace std;

int main() {
    int a = 10;
    int b = 20;

    // 1. Pointer to const int: can change where it points, cannot change *ptr_to_const
    const int* ptr_to_const = &a;
    cout << "ptr_to_const initially points to: " << *ptr_to_const << endl;
    ptr_to_const = &b; // Valid: pointer itself is not const
    cout << "ptr_to_const now points to: " << *ptr_to_const << endl;

    // 2. Const pointer to int: cannot change where it points, can change *const_ptr
    int* const const_ptr = &a;
    *const_ptr = 99; // Valid: pointed-to int is mutable
    cout << "Value of a after *const_ptr = 99: " << a << endl;

    // 3. Const pointer to const int: neither address nor value can be modified via pointer
    const int* const fixed_read_only = &b;
    cout << "Value read via fixed_read_only: " << *fixed_read_only << endl;

    return 0;
}

/*
Expected Deterministic Output:
ptr_to_const initially points to: 10
ptr_to_const now points to: 20
Value of a after *const_ptr = 99: 99
Value read via fixed_read_only: 20
*/
```

Program: `05_const_pointer_compile_error.cpp`

C++ SOURCE FILE • 05_const_pointer_compile_error.cpp

```cpp
// INTENTIONAL COMPILE-TIME ERROR DEMONSTRATION
// Program Title: Illegal Modifications with const Pointers
// Topic: Pointers and References — const Correctness Violations
// Purpose: Demonstrate why modifying *p through 'const int* p' and reassigning 'int* const
q' fail at compile time.
// Target C++ Standard: C++23
// Related Theory File: ../01_THEORY/03_const_pointers_arrays_functions_objects.md
// Program Classification: Category B — Intentional compilation-error demonstration

#include <iostream>
using namespace std;

int main() {
    int a = 10;
    int b = 20;

    const int* p = &a;
    *p = 50; // COMPILE ERROR 1: assignment of read-only location '* p'

    int* const q = &a;
    q = &b;  // COMPILE ERROR 2: assignment of read-only variable 'q'

    return 0;
}

/*
Why Compilation Fails:
1. 'p' has type 'const int*'. Dereferencing '*p' yields a 'const int' lvalue, which is non-
modifiable.
2. 'q' has type 'int* const'. The top-level 'const' qualifier applies to the pointer variable
'q' itself,
   forbidding reassignment after initialization.
See 04_const_pointer_variations.cpp for the corrected Category A version.
*/
```

Program: `06_pointer_to_pointer_and_objects.cpp`

C++ SOURCE FILE • 06_pointer_to_pointer_and_objects.cpp

```cpp
// Program Title: Pointer to Pointer and Pointers to Classroom Objects
// Topic: Pointers and References — Double Indirection & Object Pointers
// Purpose: Demonstrate int** double dereferencing and accessing a classroom 'person' object
via '→'.
// Target C++ Standard: C++23 (compatible with C++17/C++20)
// Related Theory File: ../01_THEORY/03_const_pointers_arrays_functions_objects.md
// Program Classification: Category A — Valid and well-defined program

#include <iostream>
using namespace std;

class person {
private:
    int age;

public:
    void input(int a) {
        age = a;
    }

    void display() {
        cout << "Age: " << age << endl;
    }
};

int main() {
    // Part 1: Pointer to Pointer (int**)
    int marks = 85;
    int* p = &marks;
    int** pp = &p;

    cout << "Marks via **pp: " << **pp << endl;
    **pp = 95;
    cout << "Marks after **pp = 95: " << marks << endl;

    // Part 2: Pointer to Classroom Object (person*)
    person p_obj;
    person* ptr = &p_obj;

    ptr→input(20);
    ptr→display();

    return 0;
}

/*
Expected Deterministic Output:
Marks via **pp: 85
Marks after **pp = 95: 95
Age: 20
*/
```

Program: `07_references_vs_pointers.cpp`

C++ SOURCE FILE • 07_references_vs_pointers.cpp

```cpp
// Program Title: Swapping Variables Using Pointers vs. References
// Topic: Pointers and References — Call by Address vs. Call by Reference
// Purpose: Compare pointer-based swap (explicit '&' and '*') with reference-based swap
(clean aliasing).
// Target C++ Standard: C++23 (compatible with C++17/C++20)
// Related Theory File: ../01_THEORY/05_references_and_pointer_comparison.md
// Program Classification: Category A — Valid and well-defined program

#include <iostream>
using namespace std;

void swap_by_pointer(int* x, int* y) {
    int temp = *x;
    *x = *y;
    *y = temp;
}

void swap_by_reference(int& x, int& y) {
    int temp = x;
    x = y;
    y = temp;
}

int main() {
    int a = 10;
    int b = 20;

    swap_by_pointer(&a, &b);
    cout << "After swap_by_pointer: a=" << a << ", b=" << b << endl;

    swap_by_reference(a, b);
    cout << "After swap_by_reference: a=" << a << ", b=" << b << endl;

    return 0;
}

/*
Expected Deterministic Output:
After swap_by_pointer: a=20, b=10
After swap_by_reference: a=10, b=20
*/
```

Program: `08_smart_pointers_modern_cpp.cpp`

C++ SOURCE FILE • 08_smart_pointers_modern_cpp.cpp

```cpp
// Program Title: Classroom Raw Pointer vs. Modern C++ Smart Pointers
// Topic: Pointers and References — std::unique_ptr and std::shared_ptr
// Purpose: Show how std::unique_ptr and std::shared_ptr manage a classroom 'person' object
without manual delete.
// Target C++ Standard: C++23 (compatible with C++17/C++20)
// Related Theory File: ../01_THEORY/04_memory_hazards_and_smart_pointers.md
// Program Classification: Category A — Valid and well-defined program

#include <iostream>
#include <memory>
using namespace std;

class person {
private:
    int age;

public:
    void input(int a) {
        age = a;
    }

    void display() {
        cout << "Age: " << age << endl;
    }
};

int main() {
    // 1. Exclusive ownership using std::unique_ptr
    unique_ptr<person> u_ptr = make_unique<person>();
    u_ptr->input(21);
    u_ptr->display();

    // 2. Shared ownership using std::shared_ptr
    shared_ptr<person> s_ptr1 = make_shared<person>();
    s_ptr1->input(22);
    {
        shared_ptr<person> s_ptr2 = s_ptr1;
        cout << "Shared reference count inside inner block: " << s_ptr1.use_count() << endl;
        s_ptr2->display();
    }
    cout << "Shared reference count after inner block: " << s_ptr1.use_count() << endl;

    return 0;
}

/*
Expected Deterministic Output:
Age: 21
Shared reference count inside inner block: 2
Age: 22
Shared reference count after inner block: 1
*/
```

Conceptual Questions — Module 06: Pointers and References (Questions 1–10)

Level 1 — Beginner

Q1. What is a pointer in C++? How does the value stored inside a pointer differ from the value stored inside an ordinary `int` variable?

Q2. Explain the exact roles of the address-of operator (`&`) and the dereference operator (`*`). Why does `&42` fail to compile?

Q3. Why should modern C++ programs use `nullptr` instead of `NULL` or `0` to initialize null pointers?

Level 2 — Intermediate

Q4. On a 64-bit system, `sizeof(char*)` , `sizeof(int*)` , and `sizeof(double*)` are all `8` bytes. Since all pointers have the same byte size, why does C++ require pointers to have distinct types (`char*` vs. `int*` vs. `double*`) instead of making every pointer a generic address?

Q5. Differentiate between:

1. `const int* p`
2. `int* const p`
3. `const int* const p`

State which operations (`p = &other;` and `*p = 100;`) are legal or illegal for each.

Q6. What is a `void*` (generic pointer)? List two operations that are legal on `int*` but are compile-time errors on `void*` .

Level 3 — Advanced

Q7. Distinguish clearly between a **wild pointer**, a **null pointer**, a **dangling pointer**, and a **memory leak**. Which of these can be checked safely with `if (p ≠ nullptr)` and which cannot?

Q8. State at least six technical differences between a **pointer** (`int*`) and a **reference** (`int&`). Why is an array of references (`int& arr[5]`) forbidden by the C++ standard?

Level 4 — University Examination Level

Q9. Explain the difference between **operator precedence**, **operator associativity**, and **order of evaluation (sequencing)** in C++. Why is operator precedence alone insufficient to explain why `*p++ + *p++` causes Undefined Behavior?

Q10. Compare raw owning pointers (`new` / `delete`) with modern C++ smart pointers (`std::unique_ptr` , `std::shared_ptr` , and `std::weak_ptr`). How does `std::weak_ptr` solve the circular reference problem between two `std::shared_ptr` objects?

Debugging & Find-the-Error Questions — Module 06: Pointers and References (Questions 21–26)

Level 2 — Intermediate

Q21. Identify the compile-time error in the following classroom snippet and show two ways to fix it:

CPP

```cpp
#include <iostream>
using namespace std;

class person {
public:
    int age = 20;
    void display() { cout << age << endl; }
};

int main() {
    person p;
    person* ptr = &p;
    *ptr.display(); // Find the error here
    return 0;
}
```

Q22. Identify all compile-time errors in the following `const` pointer code:

CPP

```cpp
int x = 10, y = 20;
const int* p1 = &x;
*p1 = 30;          // Line A
int* const p2 = &x;
p2 = &y;           // Line B
```

Level 3 — Advanced

Q23. Explain the fatal runtime/undefined-behavior bug in this function and rewrite it safely:

CPP

```cpp
int* create_number(int val) {
    int temp = val;
    return &temp;
}
```

Q24. Identify the bug in the following dynamic array deallocation:

```cpp
int* arr = new int[50];
arr[0] = 10;
delete arr; // What is wrong here?
```

Level 4 — University Examination Level

Q25. Identify why the following `void*` code fails to compile and correct it:

```cpp
double pi = 3.14159;
void* vp = &pi;
cout << *vp << endl;
```

Q26. Identify the subtle shallow-pointer aliasing bug when two raw pointers point to the same heap integer and both call `delete`:

```cpp
int* p1 = new int(42);
int* p2 = p1;
delete p1;
delete p2;
```

Predict the Output Questions — Module 06: Pointers and References (Questions 11–20)

Level 1 — Beginner

Q11. Predict the exact output of the following program:

```cpp
#include <iostream>
using namespace std;

int main() {
    int x = 15;
    int* p = &x;
    *p = *p + 10;
    cout << x << " " << *p << endl;
    return 0;
}
```

Q12. Predict the exact output:

```cpp
#include <iostream>
using namespace std;

int main() {
    int a = 5, b = 10;
    int& ref = a;
    ref = b;
    ref = ref + 3;
    cout << a << " " << b << endl;
    return 0;
}
```

Level 2 — Intermediate

Q13. Predict the output (testing `*p++` vs `(*p)++`):

```cpp
#include <iostream>
using namespace std;

int main() {
    int arr[3] = {10, 20, 30};
    int* p = &arr[0];
    cout << *p++ << " ";
    cout << (*p)++ << " ";
    cout << arr[0] << " " << arr[1] << " " << *p << endl;
    return 0;
}
```

Q14. Predict the output (testing `++*p` vs `*++p`):

CPP

```cpp
#include <iostream>
using namespace std;

int main() {
    int arr[3] = {10, 20, 30};
    int* p = &arr[0];
    cout << ++*p << " ";
    cout << *++p << " ";
    cout << arr[0] << " " << arr[1] << " " << *p << endl;
    return 0;
}
```

Q15. Predict the output (testing `*p--`, `--*p`, and `*--p`):

CPP

```cpp
#include <iostream>
using namespace std;

int main() {
    int arr[3] = {100, 200, 300};
    int* p = &arr[2];
    cout << *p-- << " ";
    cout << --*p << " ";
    cout << *--p << " ";
    cout << arr[0] << " " << arr[1] << " " << arr[2] << endl;
    return 0;
}
```

Level 3 — Advanced

Q16. Predict the output (testing `*(p + 1)`, `*(p - 1)`, and `*p + 1`):

CPP

```cpp
#include <iostream>
using namespace std;

int main() {
    int arr[3] = {4, 8, 12};
    int* p = &arr[1];
    cout << *(p + 1) << " " << *(p - 1) << " " << (*p + 1) << " " << *p << endl;
    return 0;
}
```

Q17. Predict the output (Double pointer `int**` manipulation):

CPP

```cpp
#include <iostream>
using namespace std;

int main() {
    int x = 10, y = 20;
    int* p = &x;
    int** pp = &p;
    **pp = 15;
    *pp = &y;
    **pp = 25;
    cout << x << " " << y << " " << *p << endl;
    return 0;
}
```

Q18. Predict the output (Pointer difference `q - p`):

CPP

```cpp
#include <iostream>
using namespace std;

int main() {
    int arr[5] = {2, 4, 6, 8, 10};
    int* p = &arr[1];
    int* q = &arr[4];
    cout << (q - p) << " " << *(p + (q - p) / 2) << endl;
    return 0;
}
```

Level 4 — University Examination Level

Q19. Predict the output (Passing pointer by value vs. pointer by reference):

CPP

```cpp
#include <iostream>
using namespace std;

int global_x = 100;

void try_reassign(int* ptr) {
    ptr = &global_x;
}

void real_reassign(int*& ptr) {
    ptr = &global_x;
}

int main() {
    int local_a = 10;
    int* p = &local_a;
    try_reassign(p);
    cout << *p << " ";
    real_reassign(p);
    cout << *p << endl;
    return 0;
}
```

Q20. Does the following program have a guaranteed output under the C++23 standard? Explain why or why not:

CPP

```cpp
#include <iostream>
using namespace std;

int main() {
    int arr[3] = {1, 2, 3};
    int* p = arr;
    int ans = (*p++) * (*p++);
    cout << ans << endl;
    return 0;
}
```

Programming Questions — Module 06: Pointers and References (Questions 27–32)

Level 2 — Intermediate

Q27. Write a C++23 program in classroom style that reverses an integer array in-place using only two pointers (`left` and `right`) and pointer arithmetic (`left++` , `right--`), without using array subscript brackets `[]` inside the loop.

Q28. Write a C++23 program with a function `void compute_stats(const int* arr, int n, int& min_val, int& max_val, int& sum)` that scans an array using a pointer to `const int` and returns the minimum, maximum, and sum through reference parameters.

Level 3 — Advanced

Q29. Write a C++23 program using a classroom `student` class (`roll` , `marks` , `input()` , `display()`) that creates an array of 3 `student` objects and iterates through them using a `student*` pointer (`ptr→input(...)` , `ptr→display()`).

Q30. Write a C++23 program that demonstrates a function returning a reference (`int& get_element(int* arr, int index)`) so that the function call can appear on the **left-hand side** of an assignment statement (`get_element(arr, 1) = 99;`).

Level 4 — University Examination Level

Q31. Write a C++23 program that counts the length of a C-style string and counts the number of vowels using a `const char*` pointer and the `*p++` idiom.

Q32. Write a C++23 program that demonstrates managing a dynamically allocated classroom `person` object first using raw `new` / `delete` and then using `std::unique_ptr<person>` .

Detailed Solutions — Conceptual Questions (Q1–Q10)

Solution to Q1 (Pointer Definition vs. Ordinary Variable)

1. **Question Restated**: What is a pointer in C++, and how does its stored value differ from an ordinary `int` variable?
2. **Concept Tested**: Memory addresses vs. direct values (`[basic.compound]`).
3. **Relevant Rule & Reasoning**:
   - An ordinary `int` variable (`int x = 25;`) stores a direct numerical quantity (`25`) in its memory bytes.
   - A pointer (`int* p = &x;`) stores the **virtual memory address** (e.g., `0x7ffeefbff5ac`) where `x` resides in memory.
4. **Common Mistake**: Confusing `p` (the address stored inside `p`) with `*p` (the integer value stored at that address).

Solution to Q2 (`&` and `*` Operators; Why `&42` Fails)

1. **Question Restated**: Explain `&` and `*` , and why `&42` fails to compile.
2. **Concept Tested**: Value categories (`lvalue` vs `prvalue`) and unary operators (`[expr.unary.op]`).
3. **Relevant Rule & Reasoning**:
   - Unary `&` requires an **lvalue** operand (an object with a persistent identity in memory) and returns its address (`T*`).
   - The integer literal `42` is a **prvalue** (pure rvalue) with no addressable storage location, so `&42` is ill-formed (Compilation Error: *lvalue required as unary* `&` *operand*).
   - Unary `*` takes a valid pointer operand (`T*`) and yields an **lvalue** referring to the target object of type `T` .

Solution to Q3 (`nullptr` vs `NULL` and `0`)

1. **Question Restated**: Why prefer `nullptr` over `NULL` or `0` ?
2. **Concept Tested**: Type safety of `std::nullptr_t` (`[lex.nullptr]`).
3. **Relevant Rule & Reasoning**:
   - `0` and `NULL` have integer types (`int` or `long`), which silently convert to integer parameters in overloaded functions (`void f(int)` vs `void f(int*)`).
   - `nullptr` has type `std::nullptr_t` , which converts implicitly to any pointer type (`T*`) but **never** to arithmetic/integer types, eliminating overload ambiguity.

Solution to Q4 (Why Pointers Have Distinct Types Despite Equal `sizeof`)

1. **Question Restated**: Why does C++ distinguish `char*`, `int*`, and `double*` if all are 8 bytes on a 64-bit machine?
2. **Concept Tested**: Dereference width/interpretation and scaled pointer arithmetic.
3. **Relevant Rule & Reasoning**:
   - **Dereferencing (`*p`)**: The compiler must know whether to read 1 byte (`char`), 4 bytes (`int`), or 8 bytes (`double`), and how to decode the bit pattern (integer vs. IEEE-754 float).
   - **Pointer Arithmetic (`p + k`)**: Adding `1` to `p` advances the raw byte address by `1 * sizeof(T)` bytes (`+1` for `char*`, `+4` for `int*`, `+8` for `double*`).

Solution to Q5 (`const int* p` vs `int* const p` vs `const int* const p`)

1. **Question Restated**: Differentiate the three `const` pointer forms.
2. **Concept Tested**: Low-level (pointed-to) `const` vs. top-level (pointer) `const`.
3. **Step-by-Step Analysis**:
   - `const int* p`: Pointer to `const int`. `p = &other` is **Legal**; `*p = 100` is a **Compile Error**.
   - `int* const p`: `const` pointer to `int`. `p = &other` is a **Compile Error**; `*p = 100` is **Legal**.
   - `const int* const p`: `const` pointer to `const int`. Both `p = &other` and `*p = 100` are **Compile Errors**.

Solution to Q6 (`void*` Generic Pointer Restrictions)

1. **Question Restated**: What is `void*` and what two operations fail on it?
2. **Concept Tested**: Incomplete type `void` (`[basic.fundamental]`).
3. **Relevant Rule & Reasoning**:
   - `void*` can hold the address of any non-const data object without knowing its concrete type.
   - **Illegal Operation 1**: Direct dereferencing (`*vp`) — fails because `sizeof(void)` and target representation are unknown.
   - **Illegal Operation 2**: Pointer arithmetic (`vp + 1` or `vp++`) — fails in ISO C++ because pointer arithmetic scales by `sizeof(T)`, which is undefined for incomplete type `void`.

Solution to Q7 (Wild vs. Null vs. Dangling Pointer vs. Memory Leak)

1. **Question Restated**: Distinguish wild, null, and dangling pointers and memory leaks.
2. **Step-by-Step Breakdown**:
   - **Wild Pointer**: Uninitialized pointer (`int* p;`) containing arbitrary stack garbage. Cannot be reliably checked with `p ≠ nullptr`!
   - **Null Pointer**: Explicitly initialized to `nullptr`. **Can** be safely checked with `if (p ≠ nullptr)`.
   - **Dangling Pointer**: Points to memory whose object lifetime has ended (e.g., after `delete p;` or returning `&local`). Cannot be detected with `p ≠ nullptr` unless `p` was manually reset to `nullptr`.

- **Memory Leak**: Heap memory allocated via `new` whose address is lost before `delete` is called.

Solution to Q8 (Pointers vs. References & Why `int& arr[5]` is Illegal)

1. **Question Restated**: Give six differences between `int*` and `int&`, and explain why arrays of references are forbidden.
2. **Concept Tested**: Reference aliasing rules (`[dcl.ref]`).
3. **Reasoning**:
   - Six differences: (1) Initialization is mandatory for `int&`, optional for `int*`; (2) `int*` can be rebound, `int&` cannot; (3) `int*` can be `nullptr`, `int&` cannot; (4) `int*` needs `*` to dereference, `int&` is automatic; (5) `&p` gives the pointer's own address, `&ref` gives the target's address; (6) `int**` exists, whereas references to references collapse.
   - **Why `int& arr[5]` is illegal**: Array elements must be objects of known size that can be contiguous in memory and addressed via pointer arithmetic (`arr + i`). References are not objects (`[basic.compound]`) and do not have their own distinct addresses.

Solution to Q9 (Precedence vs. Associativity vs. Evaluation Order / Sequencing)

1. **Question Restated**: Distinguish precedence, associativity, and evaluation order, and explain why `*p++ + *p++` is UB.
2. **Concept Tested**: Parse tree construction vs. runtime execution sequencing (`[intro.execution]`).
3. **Reasoning**:
   - **Precedence** dictates that `*p++` parses as `*(p++)`.
   - **Associativity** dictates that `++*p` parses right-to-left as `++(*p)`.
   - **Evaluation Order (Sequencing)** governs *when* the side effects of left and right operands of `+` occur at runtime. Because binary `+` leaves its left and right operands **unsequenced**, both `p++` side effects modify scalar `p` without a sequencing point between them, resulting in **Undefined Behavior**.

Solution to Q10 (Smart Pointers & Breaking Cycles with `std::weak_ptr`)

1. **Question Restated**: Compare raw pointers with `unique_ptr`, `shared_ptr`, and `weak_ptr`.
2. **Concept Tested**: RAII ownership and reference counting (`<memory>`).
3. **Reasoning**:
   - `std::unique_ptr<T>`: Exclusive ownership; zero overhead; non-copyable, movable.
   - `std::shared_ptr<T>`: Shared ownership via atomic reference counting.
   - `std::weak_ptr<T>`: If Object A holds a `shared_ptr<B>` and Object B holds a `shared_ptr<A>`, their reference counts never drop to `0` when leaving scope, causing a memory leak. Replacing one link with `std::weak_ptr` observes the target without incrementing the strong reference count, allowing deterministic destruction.

Detailed Solutions — Predict the Output & Debugging (Q11–Q26)

Solution to Q11

- **Concept Tested**: Dereference read and write (`*p = *p + 10`).
- **Dry Run**: `x` starts at `15` , `p = &x` . `*p + 10` evaluates to `15 + 10 = 25` , stored back into `*p` (`x`).
- **Guaranteed Output**:

```
TEXT
25 25
```

Solution to Q12

- **Concept Tested**: Reference assignment mutates the original referent instead of rebinding.
- **Dry Run**: `ref` binds to `a` (`5`). `ref = b;` assigns `10` into `a` (`a` is now `10` , `ref` still aliases `a`). `ref = ref + 3;` updates `a` to `13` . `b` remains `10` .
- **Guaranteed Output**:

```
TEXT
13 10
```

Solution to Q13 (`*p++` vs `(*p)++`)

- **Concept Tested**: Expression 1 (`*p++`) vs Expression 2 (`(*p)++`).
- **Step-by-Step Dry Run**:
  1. Initial: `arr = {10, 20, 30}` , `p = &arr[0]` .
  2. `cout << *p++` : Prints old `*p` (`arr[0] = 10`), advances `p` to `&arr[1]` .
  3. `cout << (*p)++` : `p` is at `&arr[1]` (`20`). Prints old `arr[1]` (`20`), increments `arr[1]` in memory to `21` . `p` stays at `&arr[1]` .
  4. Final print: `arr[0]` is `10` , `arr[1]` is `21` , `*p` is `21` .
- **Guaranteed Output**:

```
TEXT
10 20 10 21 21
```

Solution to Q14 (`++*p` vs `*++p`)

- **Concept Tested**: Expression 3 (`++*p`) vs Expression 4 (`*++p`).
- **Step-by-Step Dry Run**:

1. Initial: `arr = {10, 20, 30}`, `p = &arr[0]`.
2. `cout << ++*p`: Groups as `++(*p)`. Increments `arr[0]` from `10` to `11` and prints `11`. `p` stays at `&arr[0]`.
3. `cout << *++p`: Groups as `*(++p)`. Advances `p` to `&arr[1]` and prints `arr[1]` (`20`).
4. Final print: `arr[0]` is `11`, `arr[1]` is `20`, `*p` is `20`.

- **Guaranteed Output**:

```
TEXT
11 20 11 20 20
```

Solution to Q15 (`*p--`, `--*p`, `*--p`)

- **Concept Tested**: Expressions 7, 9, and 10.
- **Step-by-Step Dry Run**:
  1. Initial: `arr = {100, 200, 300}`, `p = &arr[2]`.
  2. `*p--`: Prints `arr[2]` (`300`), moves `p` to `&arr[1]`.
  3. `--*p`: Pre-decrements `arr[1]` from `200` to `199`, prints `199`. `p` stays at `&arr[1]`.
  4. `*--p`: Pre-decrements `p` to `&arr[0]`, prints `arr[0]` (`100`).
- **Guaranteed Output**:

```
TEXT
300 199 100 100 199 300
```

Solution to Q16 (`*(p + 1)`, `*(p - 1)`, `*p + 1`)

- **Concept Tested**: Expressions 11, 12, and 13.
- **Dry Run**: `p = &arr[1]` (`8`). `*(p + 1)` is `arr[2]` (`12`), `*(p - 1)` is `arr[0]` (`4`), `*p + 1` is `8 + 1 = 9`, and `*p` remains `8`. Note: Since C++17, `<<` chaining evaluates strictly Left-to-Right, and none of these expressions have side effects.
- **Guaranteed Output**:

```
TEXT
12 4 9 8
```

Solution to Q17 (`int**` Double Indirection)

- **Concept Tested**: Modifying `*pp` (rebinding `p`) vs `**pp` (modifying target `int`).
- **Dry Run**:
  1. `**pp = 15;` changes `x` from `10` to `15`.
  2. `*pp = &y;` changes `p` so that `p` now points to `y`.

3. `**pp = 25;` changes `y` from `20` to `25`.

- **Guaranteed Output**:

```
TEXT
15 25 25
```

Solution to Q18 (Pointer Subtraction `q - p`)

- **Concept Tested**: Subtracting two pointers into the same array returns the number of elements (`std::ptrdiff_t`) between them, **not** the raw byte difference.
- **Dry Run**: `p = &arr[1]`, `q = &arr[4]`. `q - p = 4 - 1 = 3`. `*(p + 3/2) = *(p + 1) = arr[2] = 6`.
- **Guaranteed Output**:

```
TEXT
3 6
```

Solution to Q19 (Pointer Passed by Value vs. Reference to Pointer `int*&`)

- **Concept Tested**: `void try_reassign(int* ptr)` only mutates its local copy of the pointer, whereas `void real_reassign(int*& ptr)` mutates the caller's pointer `p`.
- **Guaranteed Output**:

```
TEXT
10 100
```

Solution to Q20 (`(*p++) * (*p++)`)

- **Concept Tested**: Unsequenced side effects (`[intro.execution]`).
- **Answer**: **No guaranteed output (Undefined Behavior).** Even with parentheses around `(*p++)`, the binary multiplication operator `*` does not sequence its left operand relative to its right operand. Modifying `p` twice without sequencing is Undefined Behavior in C++23.

Solutions to Debugging Questions (Q21–Q26)

- **Q21 Solution**: In `*ptr.display();`, the dot operator `.` has higher precedence (Level 2) than unary `*` (Level 3), so the compiler parses it as `*(ptr.display())` and errors because `ptr` is a pointer, not a class object.
  - **Fix 1**: `ptr→display();`
  - **Fix 2**: `(*ptr).display();`
- **Q22 Solution**:
  - **Line A (`*p1 = 30;`)**: Compile error because `p1` is a pointer to `const int` (`const int*`).

  - **Line B (`p2 = &y;`)**: Compile error because `p2` is a `const` pointer (`int* const`).
- **Q23 Solution**: `temp` is a local automatic (stack) variable that is destroyed when `create_number` returns. Returning `&temp` yields a **dangling pointer**, and dereferencing it causes Undefined Behavior. **Fix**: Return `temp` by value (`int create_number(int val) { return val; }`) or allocate with `std::make_unique<int>(val)`.
- **Q24 Solution**: Memory allocated with array `new[]` (`new int[50]`) **must** be deallocated with array `delete[]` (`delete[] arr;`). Using scalar `delete arr;` on an array allocation is **Undefined Behavior** (`[expr.delete]`).
- **Q25 Solution**: `void* vp` cannot be dereferenced directly because `void` is an incomplete type. **Fix**: `cout << *static_cast<double*>(vp) << endl;`.
- **Q26 Solution**: Both `p1` and `p2` hold the exact same heap address. After `delete p1;`, the heap block is freed and `p2` becomes a dangling pointer. Executing `delete p2;` performs a **double deletion (double free)**, which is Undefined Behavior.

Detailed Solutions — Programming Questions (Q27–Q32)

Solution to Q27 (In-Place Array Reversal Using Two Pointers)

CPP

```cpp
#include <iostream>
using namespace std;

int main() {
    int arr[5] = {10, 20, 30, 40, 50};
    int* left = &arr[0];
    int* right = &arr[4];

    while (left < right) {
        int temp = *left;
        *left = *right;
        *right = temp;
        left++;
        right--;
    }

    for (int i = 0; i < 5; i++) {
        cout << arr[i] << " ";
    }
    cout << endl;
    return 0;
}
// Expected Output: 50 40 30 20 10
```

Solution to Q28 (Array Min, Max, and Sum via `const int*` and References)

CPP

```cpp
#include <iostream>
using namespace std;

void compute_stats(const int* arr, int n, int& min_val, int& max_val, int& sum) {
    min_val = *arr;
    max_val = *arr;
    sum = 0;

    for (int i = 0; i < n; i++) {
        int current = *(arr + i);
        if (current < min_val) min_val = current;
        if (current > max_val) max_val = current;
        sum += current;
    }
}

int main() {
    int data[5] = {12, 5, 28, 9, 16};
    int mn = 0, mx = 0, total = 0;

    compute_stats(data, 5, mn, mx, total);
    cout << "Min: " << mn << ", Max: " << mx << ", Sum: " << total << endl;
    return 0;
}
// Expected Output: Min: 5, Max: 28, Sum: 70
```

Solution to Q29 (Array of Classroom `student` Objects Accessed via Pointer)

CPP

```cpp
#include <iostream>
using namespace std;

class student {
private:
    int roll;
    int marks;

public:
    void input(int r, int m) {
        roll = r;
        marks = m;
    }

    void display() {
        cout << "Roll: " << roll << ", Marks: " << marks << endl;
    }
};

int main() {
    student list[3];
    student* ptr = list;

    for (int i = 0; i < 3; i++) {
        (ptr + i)->input(i + 1, (i + 1) * 30);
    }

    for (int i = 0; i < 3; i++) {
        ptr->display();
        ptr++;
    }

    return 0;
}
/*
Expected Output:
Roll: 1, Marks: 30
Roll: 2, Marks: 60
Roll: 3, Marks: 90
*/
```

Solution to Q30 (Function Returning an Lvalue Reference `int&`)

CPP

```cpp
#include <iostream>
using namespace std;

int& get_element(int* arr, int index) {
    return *(arr + index);
}

int main() {
    int arr[3] = {10, 20, 30};
    get_element(arr, 1) = 99; // Modifies arr[1] directly!

    cout << arr[0] << " " << arr[1] << " " << arr[2] << endl;
    return 0;
}
// Expected Output: 10 99 30
```

Solution to Q31 (String Length and Vowel Count Using `const char*` and `*p++`)

CPP

```cpp
#include <iostream>
using namespace std;

int main() {
    const char* text = "Mathematics";
    const char* p = text;
    int length = 0;
    int vowels = 0;

    while (*p != '\0') {
        char ch = *p++;
        length++;
        if (ch == 'a' || ch == 'e' || ch == 'i' || ch == 'o' || ch == 'u' ||
            ch == 'A' || ch == 'E' || ch == 'I' || ch == 'O' || ch == 'U') {
            vowels++;
        }
    }

    cout << "Length: " << length << ", Vowels: " << vowels << endl;
    return 0;
}
// Expected Output: Length: 11, Vowels: 4
```

Solution to Q32 (Classroom `person` via Raw `new` / `delete` vs `std::unique_ptr`)

CPP

```cpp
#include <iostream>
#include <memory>
using namespace std;

class person {
private:
    int age;

public:
    void input(int a) {
        age = a;
    }

    void display() {
        cout << "Age: " << age << endl;
    }
};

int main() {
    // 1. Classroom raw pointer approach
    person* raw_p = new person;
    raw_p->input(20);
    raw_p->display();
    delete raw_p;
    raw_p = nullptr;

    // 2. Modern C++23 smart pointer approach
    unique_ptr<person> smart_p = make_unique<person>();
    smart_p->input(25);
    smart_p->display();

    return 0;
}
/*
Expected Output:
Age: 20
Age: 25
*/
```